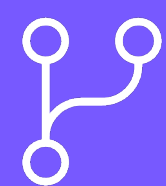
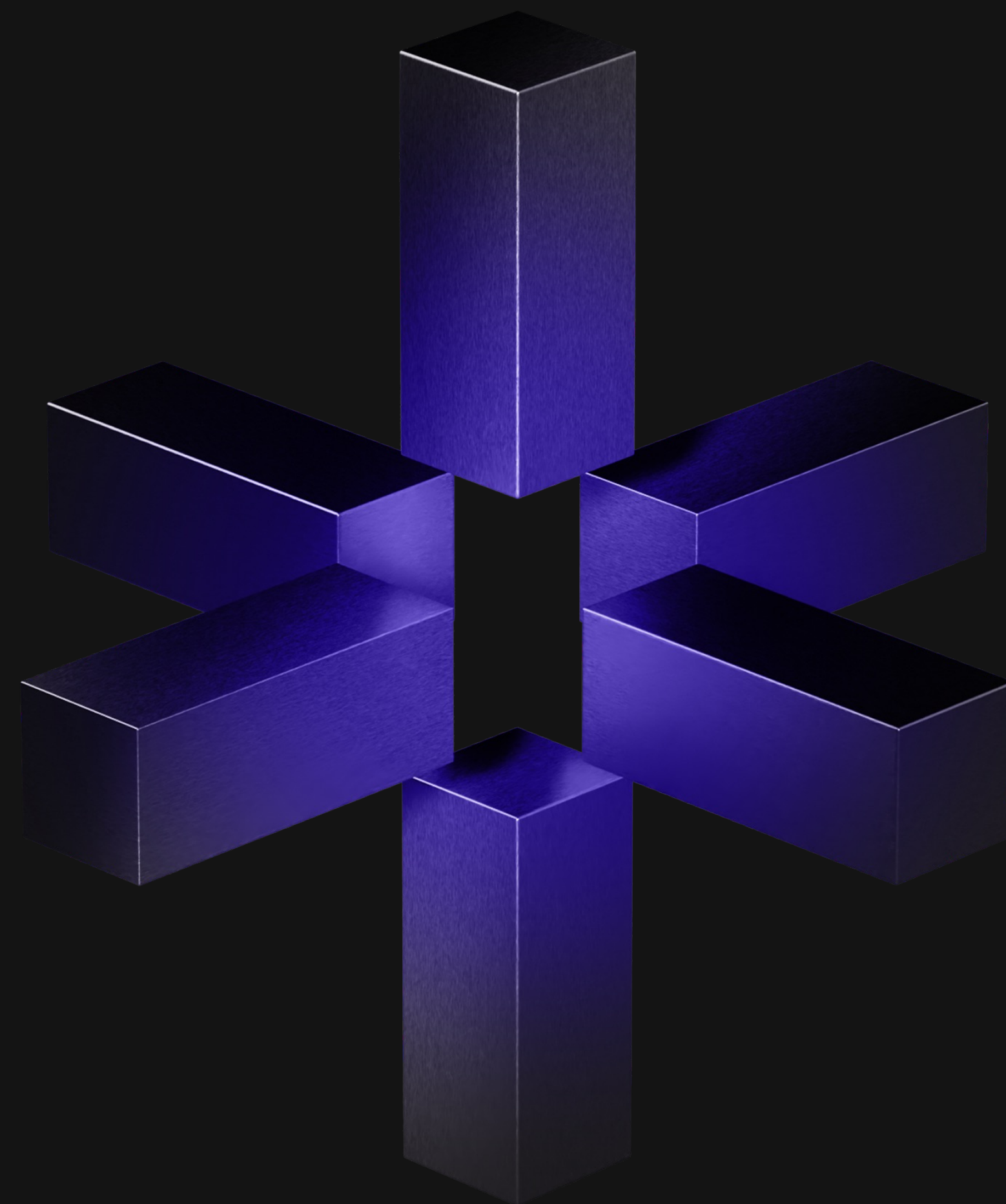


# Инструменты разработки



ML

Production ML (Машинное обучение в продакшене)

01

Алексей Борисов

# План лекции



→ Основы работы в командной строке Linux

→ SSH и удалённая разработка

→ Система контроля версий  
и совместная разработка с Git

→ Структура проекта и принципы организации кода

→ Работа в VS Code как основной IDE курса

# План лекции



→ Организация Python-проектов

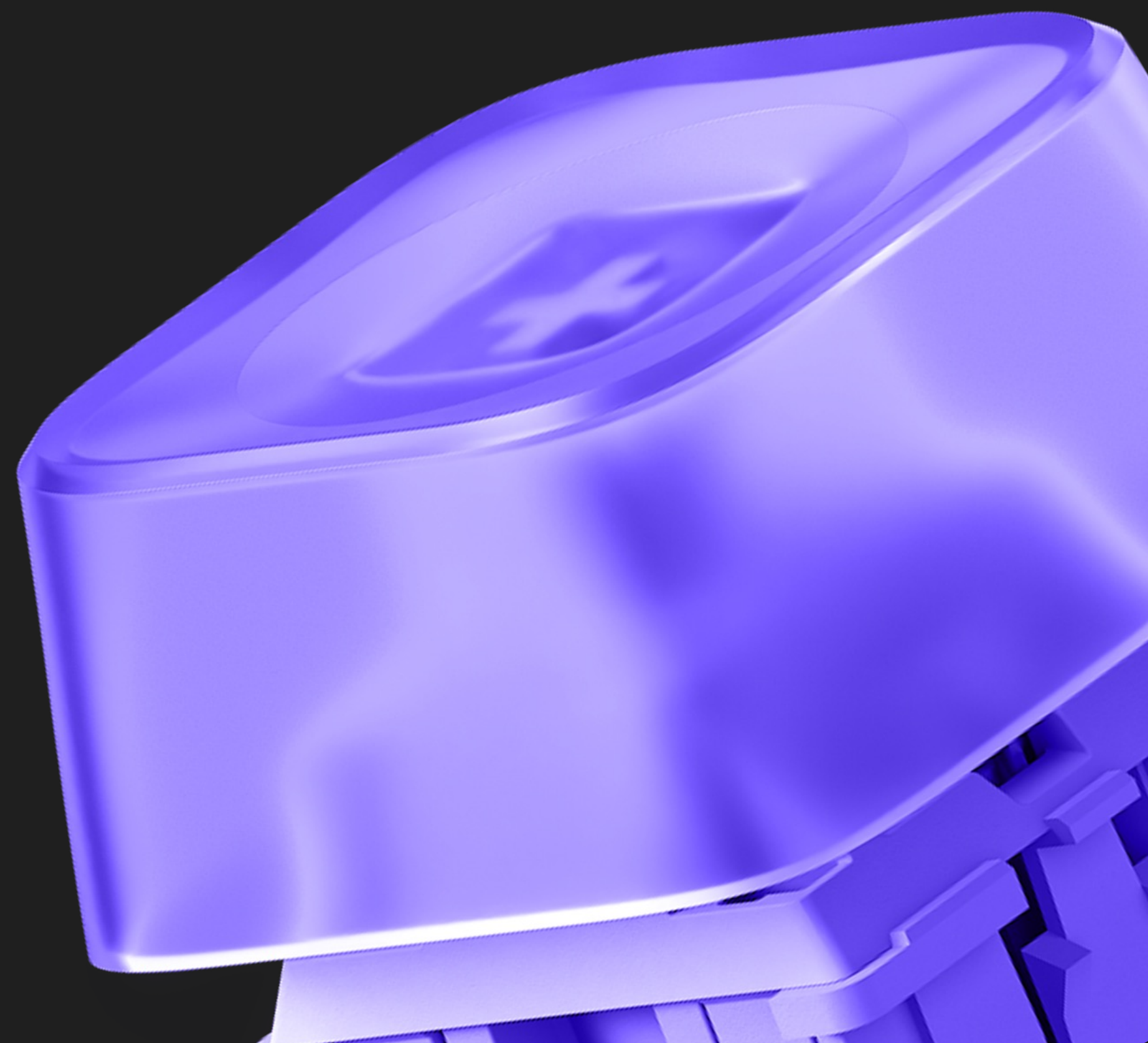
→ Запуск и отладка Python-сервиса

→ Контейнеризация с Docker

→ Docker Compose



# Работа в командной строке Linux





# Командная строка

**Shell** — это интерфейс для общения с операционной системой через команды.



- Выполнение команд без графического интерфейса
- Автоматизация задач с помощью скриптов
- Работа на удалённых серверах через SSH
- Управление файлами и процессами в системе



PowerShell



**BASH**  
THE BOURNE-AGAIN SHELL

# Навигация по файловой системе

```
> pwd
/root/week-2-development-tools

> ls -l
README.md  project_structure.ini  test

> cd test

> tree
.
├── file1.txt
├── nested1
│   ├── file2.yaml
│   └── file3.bin
└── nested2
    ├── file4.txt
    └── file5.py

> find . -type f -name "*.txt"
./nested2/file4.txt
./file1.txt
```

- **pwd** выводит текущую рабочую директорию.
- **ls** выводит список элементов в директории.
- **cd** — переход в директорию.
- **tree** выводит дерево каталогов.
- **find** — поиск файлов по заданному критерию.

# Работа с файлами и директориями

```
> touch file6.py
> mkdir nested3
> cp file6.py nested3
> cp -r nested2 nested3
> mv nested3/nested2 nested3/nested4
```

```
> tree
```

```
.
├── file1.txt
├── file6.py
├── nested1
│   ├── file2.yaml
│   └── file3.bin
├── nested2
│   ├── file4.txt
│   └── file5.py
└── nested3
    ├── file6.py
    └── nested4
        ├── file4.txt
        └── file5.py
```

```
> rm -r file6.py nested3/nested4
```

- **touch** создаёт пустой файл.
- **mkdir** создаёт новую директорию.
- **cp** копирует файл или директорию.
- **mv** перемещает или переименовывает файл или директорию.
- **rm** удаляет файл или директорию.



# Просмотр содержимого файлов

```
> cat file1.txt
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.

> head -n 2 file1.txt
Beautiful is better than ugly.
Explicit is better than implicit.

> tail -n 1 file1.txt
Complex is better than complicated.

> less file1.txt

> wc file1.txt
  4  20 132 file1.txt

> vim file1.txt
```

- **cat** выводит содержимое файла целиком.
- **head** выводит первые строки файла.
- **tail** выводит последние строки файла.
- **less** — интерактивный просмотр содержимого файла.
- **wc** — подсчёт строк, слов и символов в тексте.
- **vim/nano** — консольный текстовый редактор.

# Комбинирование команд, пайпы и перенаправления

```
> pwd; ls
/root/week-2-development-tools
README.md  project_structure.ini  test

> mkdir test2 && cd test2

> cd test3 || echo "No such directory"
No such directory

> head -n 2 file1.txt | wc
      2      10      65

> cat file1.txt file1.txt > file7.txt

> A=32
> B=10
> echo $(( $A+$B ))
42

> test $A -gt $B && echo $A || echo $B
32
```

- Команды можно объединять через `;`, `&&` и `||`, чтобы выполнять их последовательно или с условием.
- Вывод одной команды можно направить на ввод другой через пайп (`|`).
- Можно переопределить команды `stdin`, `stdout` и `stderr` на другой файловый дескриптор, файл. Это делается с помощью `<`, `>` и `2>`.
- Можно задавать переменные с помощью `VAR=value`.
- Shell позволяет писать полноценные скрипты: есть синтаксис для задания функций, циклов, проверки условий и т. д.

# Обработка текста и строк

```
> echo "Hello, World!"
Hello, World!

> printf "Name: %s, Age: %d\n" \
> "Alice" 25
Name: Alice, Age: 25

cat file1.txt | grep "lex"
Simple is better than complex.
Complex is better than complicated.

> cut -f5 -d " " file1.txt | head -n 2
ugly.
implicit.

> awk '{ for(i=1;i<=NF;i++) if($i=="Complex") c++ }
END { print c }' file1.txt
1

> echo "hello world" | tr 'a-z' 'A-Z'
HELLO WORLD
```

- **echo** выводит аргумент или значение переменной.
- **printf** — форматированный вывод текста.
- **grep** — поиск и фильтрация строк по шаблону.
- **cut** — извлечение указанных полей или диапазонов символов из строк.
- **awk** — обработка и форматирование строк по шаблонам.
- **tr** — замена или удаление СИМВОЛОВ.



# Обработка текста и строк

```
> sed 's/better/worse/' file1.txt | head -n 1
Beautiful is worse than ugly.

> echo "Hello, World!" | rev
!dlroW ,olleH

> echo -e "B\nA\nC" | sort
A
B
C

> echo -e "B\nA\nA\nC\nB" | sort | uniq -c
      2 A
      2 B
      1 C

> echo '{"name":"Alice","age":30,"city":"Paris"}' |
jq '.name'
"Alice"
```

- **sed** — потоковый редактор для замены и изменения текста.
- **rev** — разворот строк в обратном порядке.
- **sort** — сортировка строк.
- **uniq** — удаление повторяющихся строк.
- **jq** — фильтрация и обработка json.

# Работа с процессами и системой

```
> ps -aux
```

```
...
```

```
> top
```

```
> kill -9 <PID>
```

```
> pkill -f python
```

```
> df -h
```

| Filesystem | Size | Used | Avail | Use% | Mounted on |
|------------|------|------|-------|------|------------|
| udev       | 28G  | 0    | 28G   | 0%   | /dev       |
| tmpfs      | 5.5G | 128M | 5.4G  | 3%   | /run       |
| /dev/vda1  | 500G | 23G  | 478G  | 5%   | /          |

```
> du -h
```

```
4.0K ./nested1
```

```
> free -h
```

|       | total | used  | free | shared | buff/cache | available |
|-------|-------|-------|------|--------|------------|-----------|
| Mem:  | 54Gi  | 3.2Gi | 32Gi | 127Mi  | 19Gi       | 50Gi      |
| Swap: | 0B    | 0B    | 0B   |        |            |           |

- **ps** — список активных процессов.
- **top/htop** — интерактивный мониторинг процессов и ресурсов.
- **kill/pkill** — отправка сигнала процессу.
- **df** — свободное и занятое место на дисках.
- **du** — размер файлов и директорий.
- **free** — использование оперативной памяти и swap.



# Управление правами и пользователями

```
> whoami
root

> id
uid=0(root) gid=0(root) groups=0(root)

> ls -lah file1.txt
-rw-r--r-- 1 root root 132 Aug 18 00:59 file1.txt

> chown alice file1.txt
> chmod +x file1.txt

> ls -lah file1.txt
-rwxr-xr-x 1 alice root 132 Aug 18 00:59 file1.txt
```

- **chown** — изменить владельца файла или директории.
- **chmod** — изменить права доступа к файлу или директории.
- **whoami** — текущий пользователь.
- **id** — информация о пользователе и группах.

# Другие полезные команды

```
> clear

> man ls

> sudo vim ~/.ssh/authorized_keys

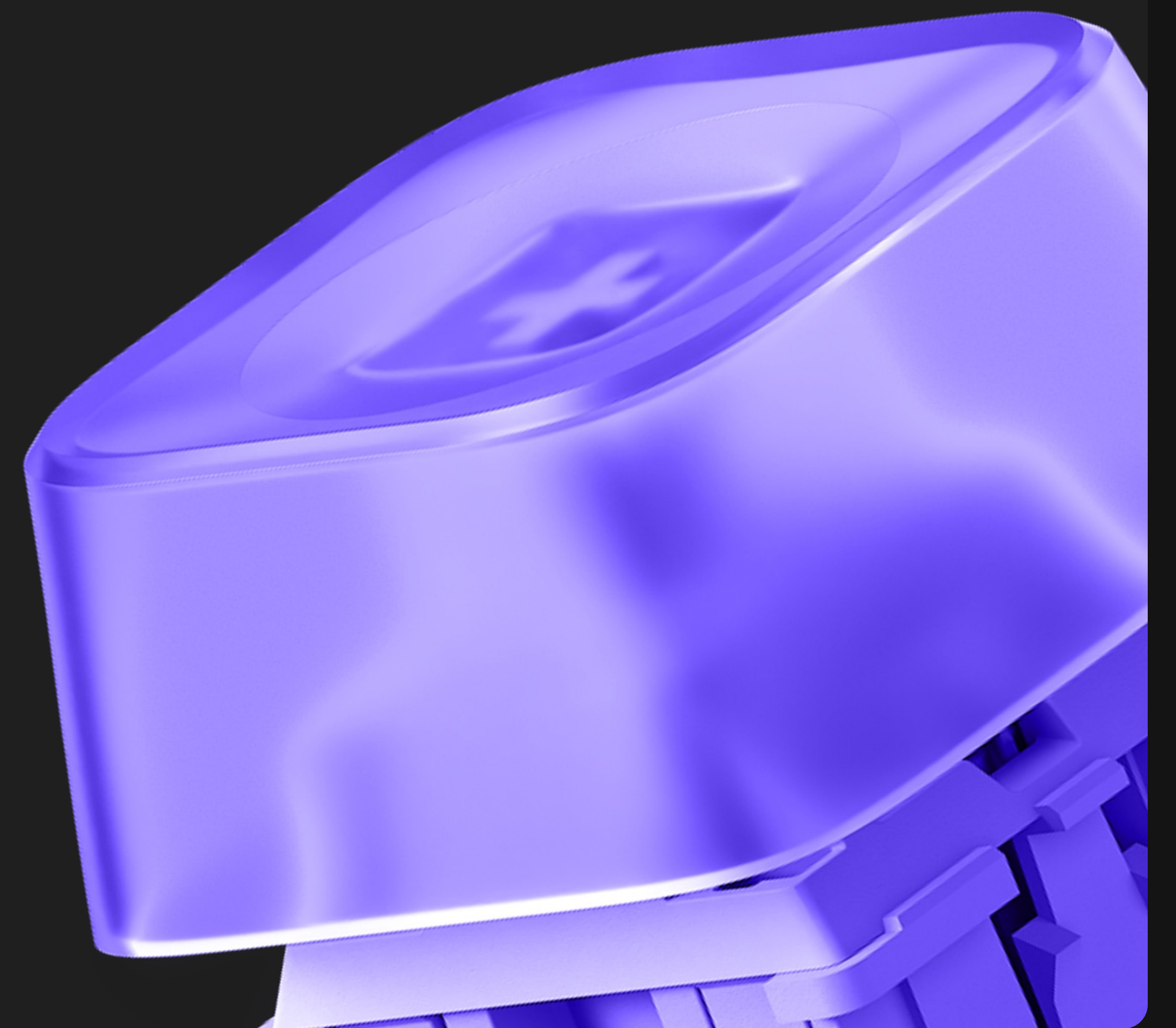
> date
Mon Aug 18 02:32:25 MSK 2025

> tar -czf archive.tar.gz file* dir*
> tar -xf archive.tar.gz

> source ~/.bashrc
```

- **clear** — очистка терминала.
- **man** — просмотр справочника по команде.
- **sudo** — выполнение команды с правами суперпользователя (root).
- **date** — показ текущей даты и времени.
- **tar, zip/unzip/gzip/bzip/zcat** — основные утилиты для работы с архивами.
- **source** — выполнение команд из файла в текущей оболочке.

# SSH и удалённая разработка





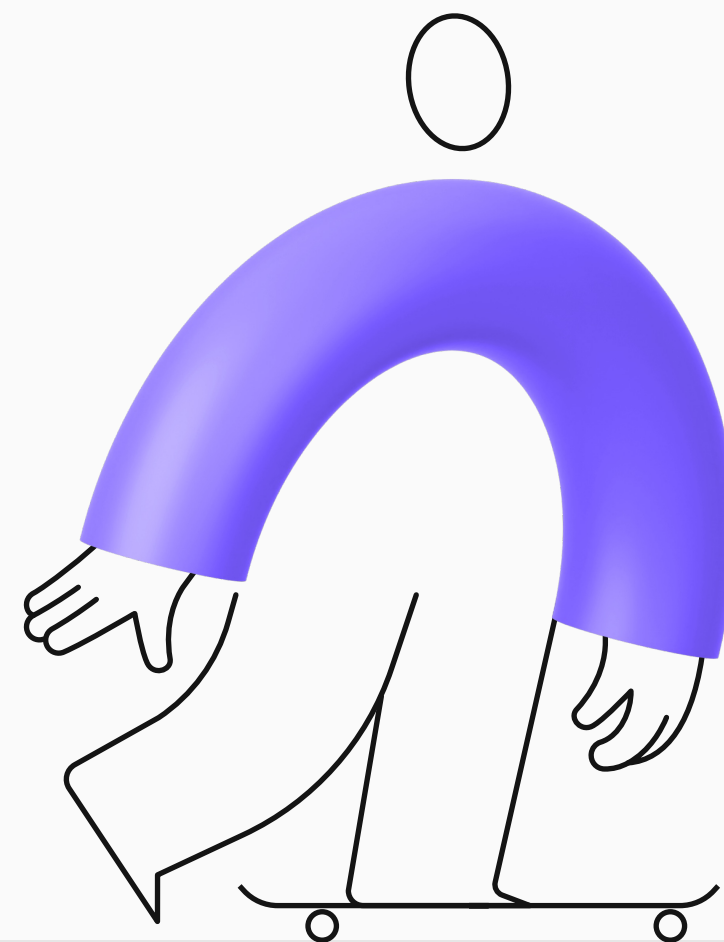
# Удалённая разработка



**Виртуальная машина (ВМ)** — компьютерная система, эмулирующая возможности каких-либо вычислительных комплексов.



**Удалённая разработка** — работа с кодом и сервисами на машине, которая физически находится не у вас, а в дата-центре или облаке.



# Удалённая разработка

- **Доступ к мощным ресурсам:** серверные CPU, GPU и увеличенный объём памяти.
- **Единая среда разработки:** окружение виртуальной машины совпадает с production, что снижает риск несовместимостей.
- **Непрерывность работы:** виртуальная машина доступна в любое время, независимо от состояния локального компьютера.
- **Безопасность и изоляция:** код, данные и ключи хранятся на удалённом сервере; локальная машина используется только для доступа.

# Удалённая разработка

**SSH (Secure Shell)** — протокол для безопасного доступа к удалённым системам. Работает поверх TCP, по умолчанию использует порт 22.



→ Шифрование трафика (в отличие от telnet)

→ Защита от подслушивания и атак

→ Аутентификация (по паролю или ключу)

→ Туннелирование и проброс портов





# Аутентификация в SSH

## При помощи пароля

- ✓ **Простота:** не требует дополнительной настройки.
- ✗ Подвержена брутфорс-атакам при слабых паролях (и не только).
- ✗ Требуется регулярного ввода пароля, неудобно для автоматизации.

## При помощи ключей

- ✓ **Устойчива к брутфорсу** (подбор ключа невозможен).
- ✓ Секретная часть вообще не передаётся по сети.
- ✓ Подходит для **автоматизации**.
- ✗ **Требуется настройки:** генерации ключей и добавления публичного ключа на сервер.

# SSH: основные команды

Подключение к серверу

```
ssh user@hostname
```

Указание порта

```
ssh -p <port> user@hostname
```

Копирование файлов

```
scp file.txt user@hostname:/path/  
scp user@hostname:/path/file.txt .
```

Копирование директорий

```
scp -r project/ user@hostname:/path/
```

Исполнение команды на сервере

```
ssh user@hostname <command>
```

Генерация пары ключей

```
ssh-keygen -t rsa -b 4096 -C "comment"
```

Отправка публичного ключа на сервер

```
ssh-copy-id user@host
```

Явное указание ключа

```
ssh -i ~/.ssh/id_rsa user@hostname
```

Просмотр имеющихся ключей

```
ssh-add -l
```

Добавление ключа

```
ssh-add ~/.ssh/id_rsa
```

# Port forwarding

SSH позволяет перенаправлять сетевой трафик через защищённый канал, создавая туннели между клиентом и сервером.

**Локальный проброс (local forwarding)** направляет локальный порт на сервер  
`ssh -L 8080:localhost:80 user@server.`

Любой запрос к `localhost:8080` на клиенте перенаправляется на `server:80`.

**Пример:** доступ к сервисам или базам данных, недоступных напрямую.

**Удалённый проброс (remote forwarding)** направляет порт сервера на локальный компьютер  
`ssh -R 9090:localhost:3000 user@server.`

Любой доступ к `server:9090` перенаправляется на `localhost:3000` клиента.

**Пример:** подключение дебаггера к процессу, запущенному удалённо.

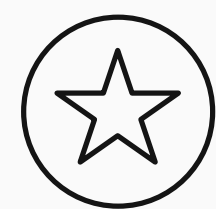


# Конфигурация SSH

SSH позволяет хранить настройки подключения в файле `~/.ssh/config`, что упрощает работу с удалёнными машинами.

```
Host myserver                                # указание алиаса
  HostName example.com                      # адрес сервера
  User user                                 # имя пользователя
  Port 5000                                 # порт SSH
  ForwardAgent Yes                          # проброс ssh-ключей
  ServerAliveInterval 60                   # keep-alive
  IdentityFile ${HOME}/.ssh/id_rsa         # приватный ключ
  LocalForward 8080 localhost:9096         # проброс локального порта
  RemoteForward 5502 localhost:5502       # проброс удалённого порта
```

# Работа с долгоживущими процессами на сервере



Если SSH-сессия закрывается или прерывается по любой причине, запущенные в ней команды останавливаются.

`nohup` позволяет запускать команду так, чтобы она **продолжала работать после выхода из сессии**.

## ПРИМЕР

`nohup python train.py > output.log 2>&1 &` — запуск в фоне, `> output.log` и `2>&1` — перенаправление вывода.

# Работа с долгоживущими процессами на сервере

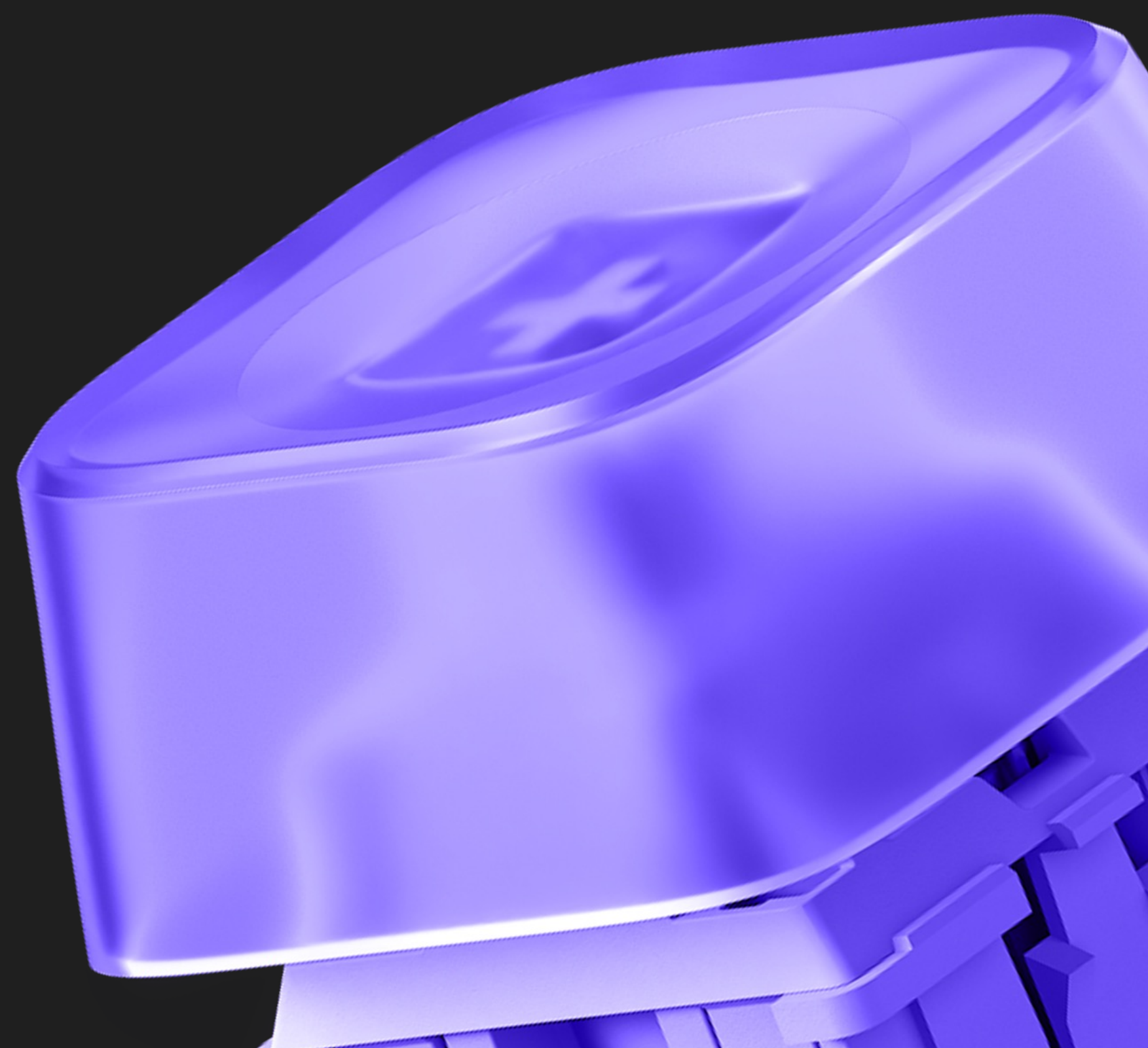
`tmux/screen` — терминальные мультиплексоры, позволяющие иметь несколько сессий, вкладок и окон, которые не зависят от подключения.



```
tmux new -s session_name      # создать новую сессию
tmux attach -t session_name    # подключиться к существующей
tmux detach                    # отсоединиться, процесс продолжает работать
tmux ls                         # список сессий
```



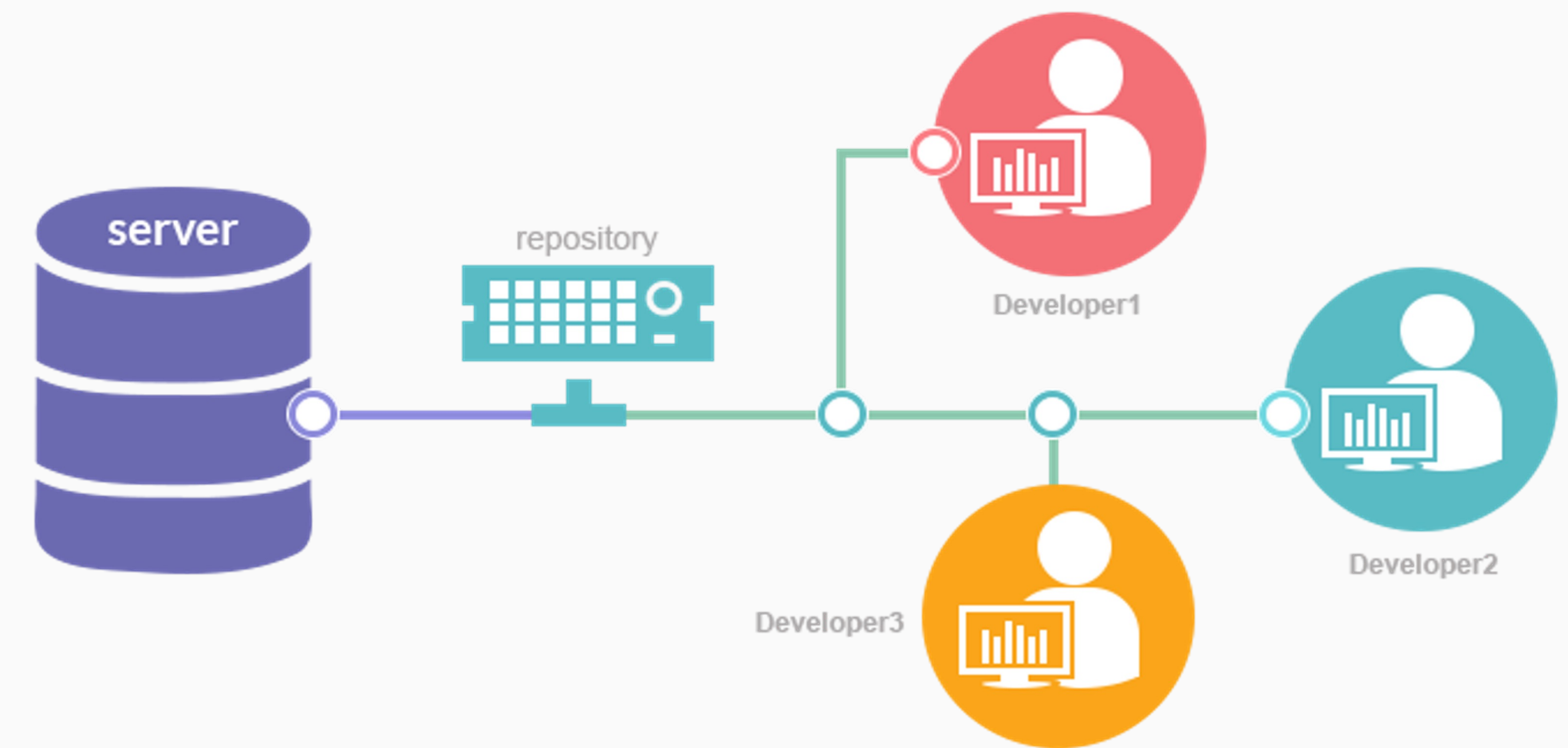
# Git и совместная разработка





# Система контроля версий

- **История изменений** — возможность вернуться к любой предыдущей версии кода.
- **Безопасность работы** — эксперименты в ветках не влияют на основную версию.
- **Совместная разработка** — несколько человек могут работать над проектом параллельно.
- **Прозрачность** — видно, кто и какие изменения внёс; удобно для ревью.
- **Интеграция с автоматизацией** — CI/CD, тестирование и деплой напрямую из репозитория.



# Git: основные сущности

## Репозиторий (repository)

- Хранит весь код проекта и историю изменений.
- Может быть локальным (на вашей машине) и удалённым (GitHub/GitLab).

## Remote (удалённый репозиторий)

- Общий репозиторий для совместной работы.

## Коммит (commit)

- Снимок текущего состояния файлов.
- Имеет уникальный SHA-хеш и сообщение с описанием изменений.

## Staging area / индекс

- Промежуточное место, где файлы готовятся к коммиту.
- Позволяет коммитить только выбранные изменения.

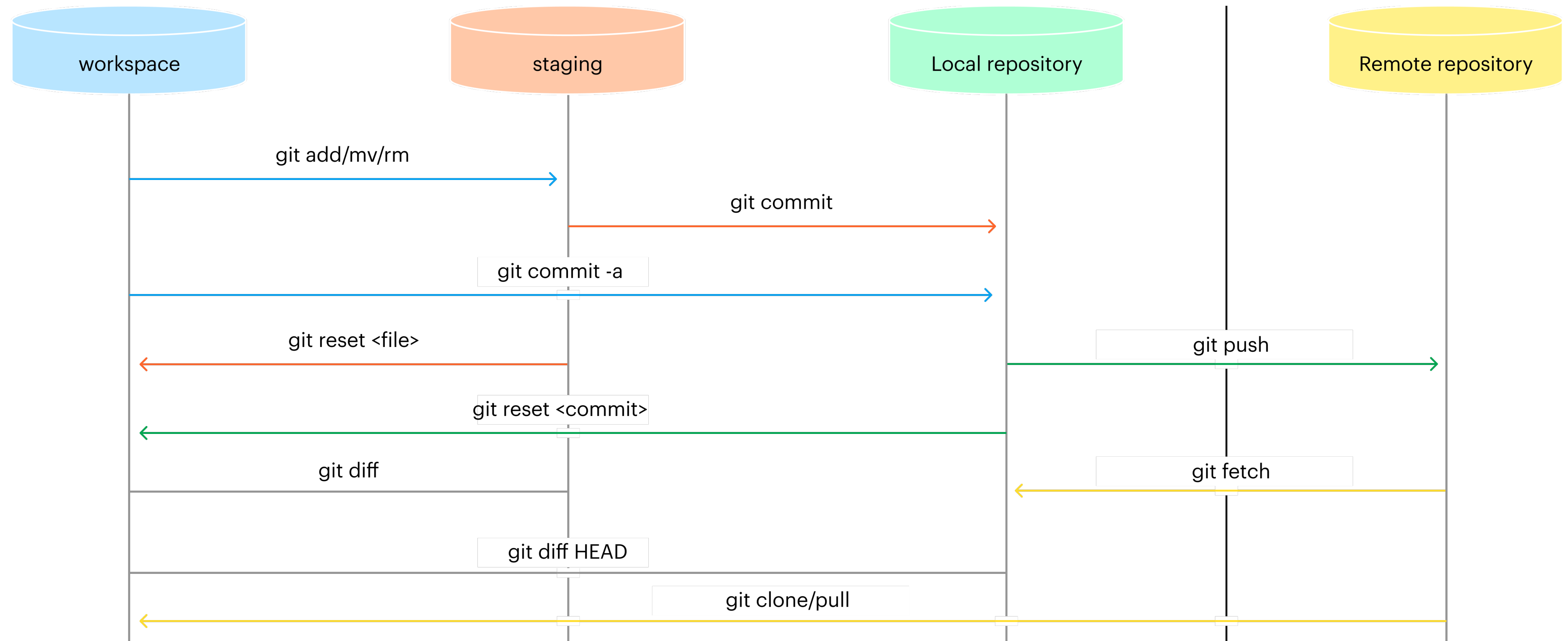
## Ветка (branch)

- Отдельная линия разработки.
- Позволяет экспериментировать и параллельно работать с кодом.

## HEAD

- Указывает на текущие коммит/ветку, с которыми вы работаете.

# Git: основные сущности





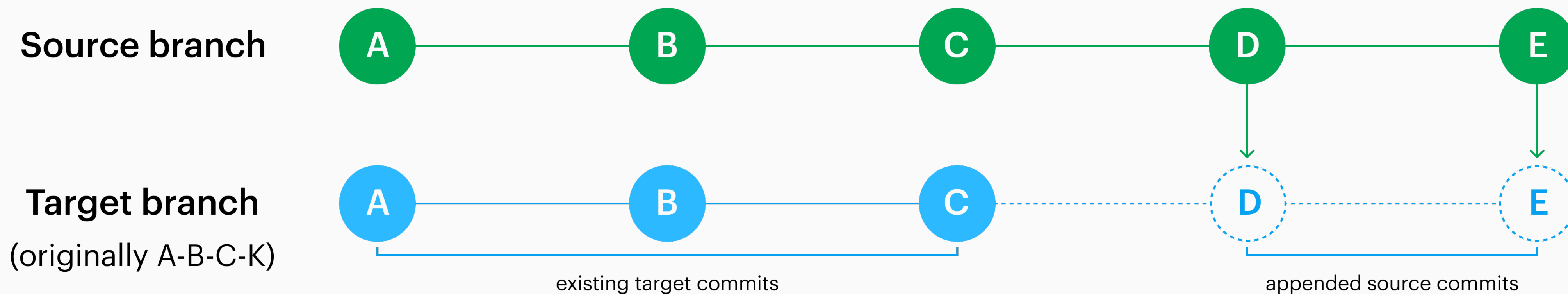
# Git: основные команды

|                                              |                                           |
|----------------------------------------------|-------------------------------------------|
| Инициализация нового репозитория             | <code>git init</code>                     |
| Клонирование существующего репозитория       | <code>git clone &lt;url&gt;</code>        |
| Просмотр текущего состояния                  | <code>git status</code>                   |
| Просмотр истории коммитов                    | <code>git log</code>                      |
| Добавление изменений в индекс (staging area) | <code>git add &lt;file&gt;</code>         |
| Создание коммита                             | <code>git commit -m "message"</code>      |
| Просмотр изменений                           | <code>git diff</code>                     |
| Переключение ветки                           | <code>git checkout &lt;branch&gt;</code>  |
| Создание новой ветки                         | <code>git branch &lt;branch&gt;</code>    |
| Удаление ветки                               | <code>git branch -d &lt;branch&gt;</code> |

# Git: основные команды

|                                              |                                                      |
|----------------------------------------------|------------------------------------------------------|
| Переименование ветки                         | <code>git branch -m &lt;old&gt; &lt;new&gt;</code>   |
| Создание ветки с переключением               | <code>git checkout -b &lt;branch&gt;</code>          |
| Отправка изменений на удалённый репозиторий  | <code>git push</code>                                |
| Получение изменений с удалённого репозитория | <code>git fetch</code>                               |
| Получение и автоматическое слияние изменений | <code>git pull</code>                                |
| Просмотр удалённых репозитория               | <code>git remote -v</code>                           |
| Добавление нового удалённого репозитория     | <code>git remote add &lt;name&gt; &lt;url&gt;</code> |
| Слияние ветки в текущую                      | <code>git merge &lt;branch&gt;</code>                |
| Ребейз текущей ветки на другую ветку         | <code>git rebase &lt;branch&gt;</code>               |

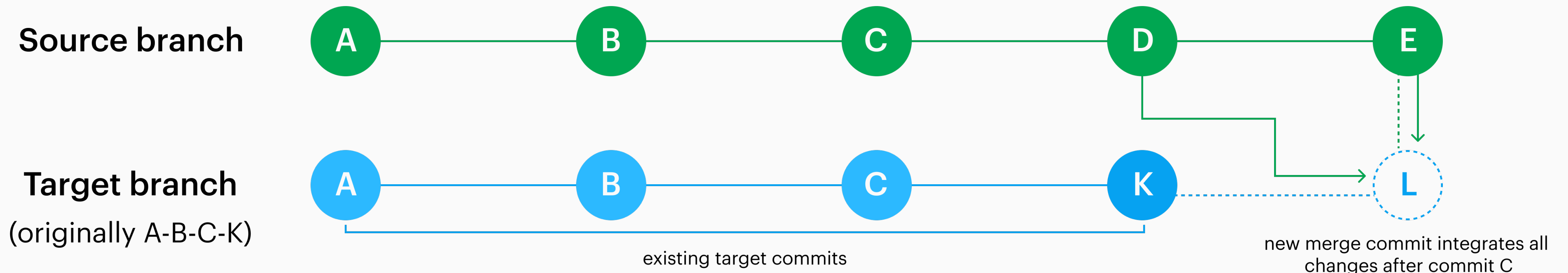
# Слияние веток: fast-forward



```
git checkout target_branch  
git merge --ff-only source_branch
```



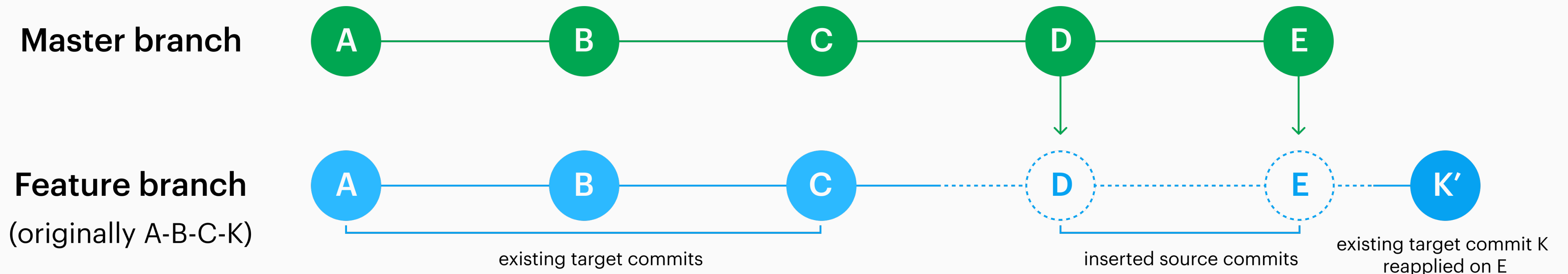
# Слияние веток: merge



```
git checkout target_branch  
git merge source_branch
```



# Слияние веток: rebase



```
git checkout feature_branch
git rebase master
```

Часто при работе в **feature\_branch** периодически производят **rebase** на основную ветку, чтобы поддерживать её актуальность и минимизировать число конфликтов.

# Платформы для совместной разработки



# Соглашения наименований

**Conventional Commits** — один из вариантов соглашения о названии коммитов.

Наименования веток должны быть короткими, читаемыми и отражать цель.

## Формат

`<type>[optional scope]: <description>`

## Примеры

**feature/login-page** —  
новая функциональность.

**hotfix/crash-on-start** —  
срочное исправление.

**release/video-service-21** —  
релизная ветка.

## Примеры

**feat(parser) : add CSV support**  
**refactor: remove outdated toggles**

Поддерживает читаемость истории проекта.

Облегчает совместную работу нескольких разработчиков.

Упрощает автоматизацию.



# Pre-commit hooks

**Pre-commit hooks** — это скрипты, которые выполняются до того, как изменения попадут в репозиторий.



```
repos:
-   repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v2.3.0
    hooks:
    -   id: check-yaml
    -   id: end-of-file-fixer
    -   id: trailing-whitespace
-   repo: https://github.com/psf/black
    rev: 22.10.0
    hooks:
    -   id: black
```

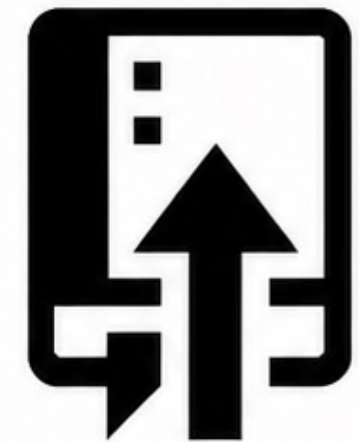
Что можно делать  
с помощью pre-commit hooks

- Проверять форматирование и стиль кода (линтеры, black, flake8).
- Запускать юнит-тесты или другие проверки.
- Проверять соответствие сообщений коммитов соглашениям (например, Conventional Commits).
- Предотвращать добавление чувствительных данных (ключей, паролей).

# IN CASE OF FIRE



1. git commit



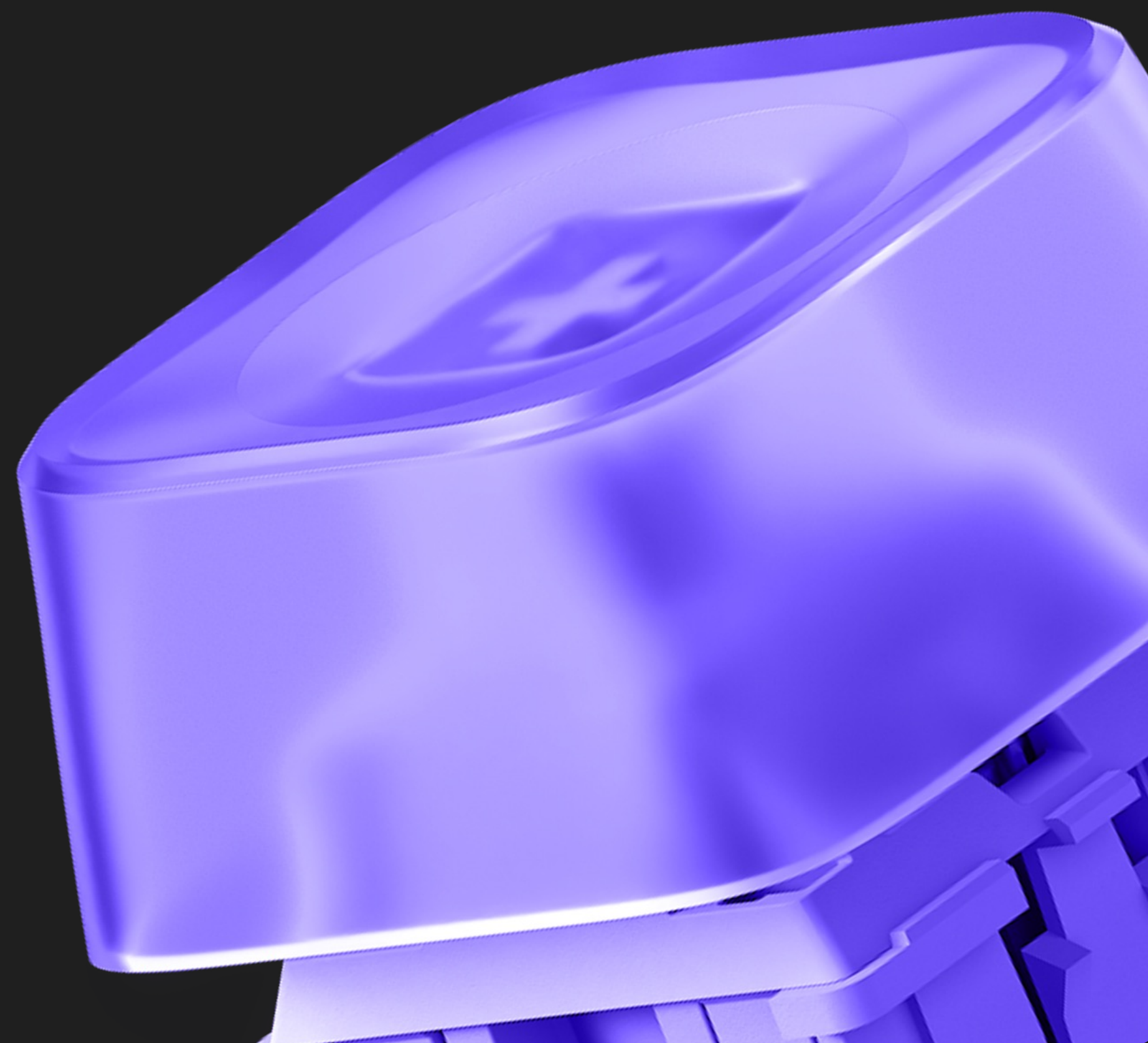
2. git push



3. git out



# Структура проекта



# Структура проекта

```
ml_project/
├── README.md           # Корневая папка проекта
├── pyproject.toml       # Краткое описание проекта и инструкция по запуску
├── requirements.txt     # Описание зависимостей и настроек сборки
├── .gitignore          # (Опционально) зависимости для pip
├── data/               # Исключения для Git (кеши, временные файлы)
│   ├── raw/            # Данные (не всегда храним в git)
│   └── processed/      # Сырые данные
├── notebooks/          # Предобработанные данные
├── src/                # Jupyter Notebook'и для экспериментов
│   ├── __init__.py     # Исходный код проекта
│   ├── model.py        # Определение модели
│   └── train.py        # Скрипт обучения модели
├── models/             # Сохранённые модели (артефакты обучения)
├── tests/              # Автоматические тесты
│   ├── __init__.py
│   ├── test_preprocessing.py # Тесты функций предобработки
│   └── test_model.py     # Тесты модели
├── scripts/            # Утилиты и вспомогательные скрипты
│   ├── init.sh         # Первоначальная настройка окружения
│   └── download_data.py # Пример: загрузка данных с сервера
├── configs/            # Конфигурационные файлы (yaml/json)
│   └── train_config.yaml # Параметры обучения
├── docs/               # Документация проекта
│   └── architecture.md # Описание архитектуры решения
```

→ Упорядочивает файлы и папки проекта.

→ Позволяет быстро ориентироваться в коде.

→ Разделяет код, тесты, данные и документацию.

→ Облегчает воспроизводимость и стандартизацию.

# Структура проекта

**Структура проекта** — это не только папки и файлы.

**Организация версий** — [Semantic Versioning](#) (MAJOR.MINOR.PATCH) помогает отслеживать изменения и совместимость.

**Конфигурация** — pyproject.toml, requirements.txt улучшают воспроизводимость проекта.

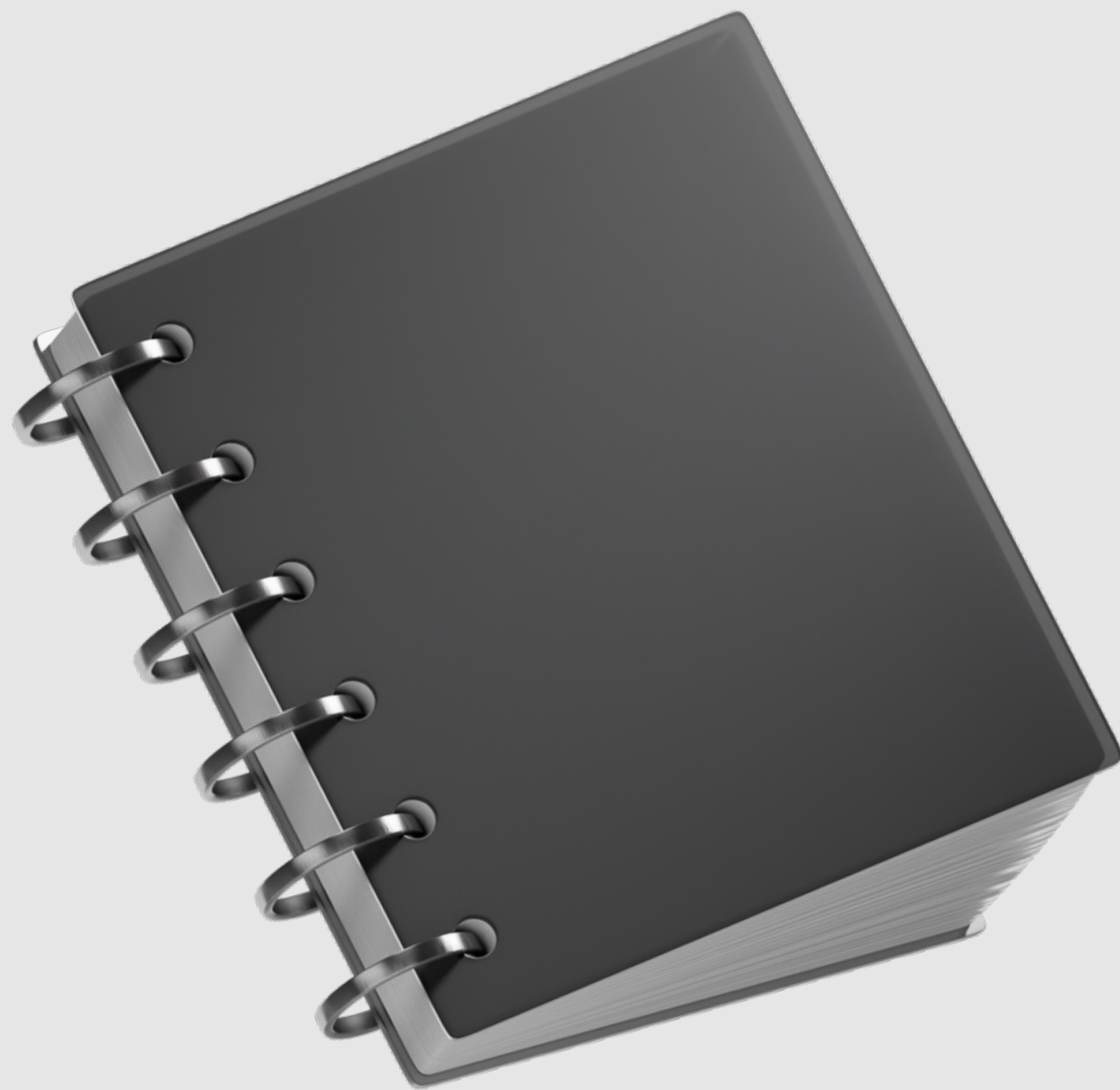
**Документация** — README и docs/ объясняют структуру, назначение компонентов и правила работы.

**Кодовые стандарты** — единый стиль, линтеры, форматирование, соглашения по именованию.

**Автоматизация и тесты** — тесты, CI/CD и pre-commit hooks поддерживают качество и стабильность проекта.

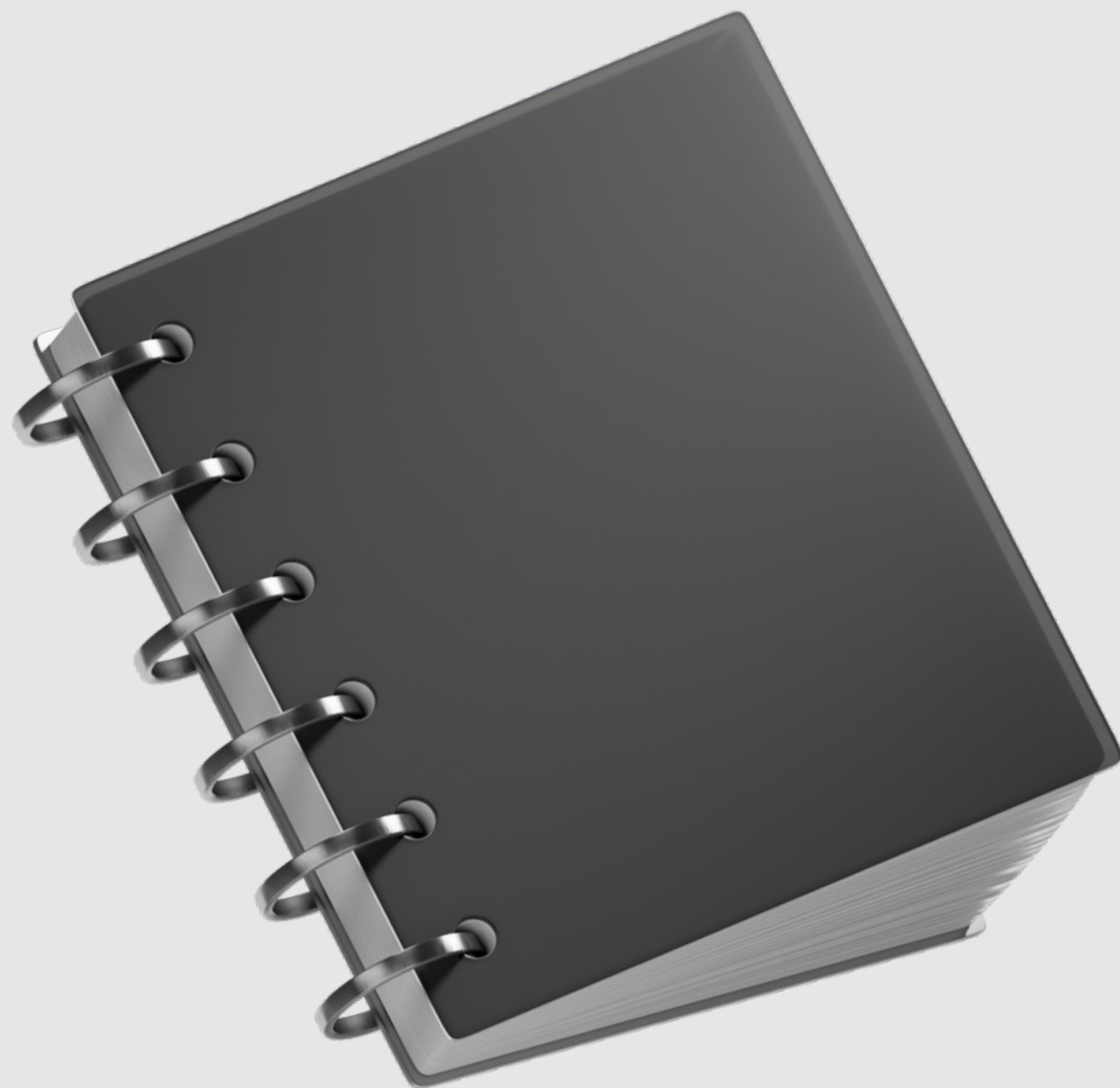


# Структура проекта



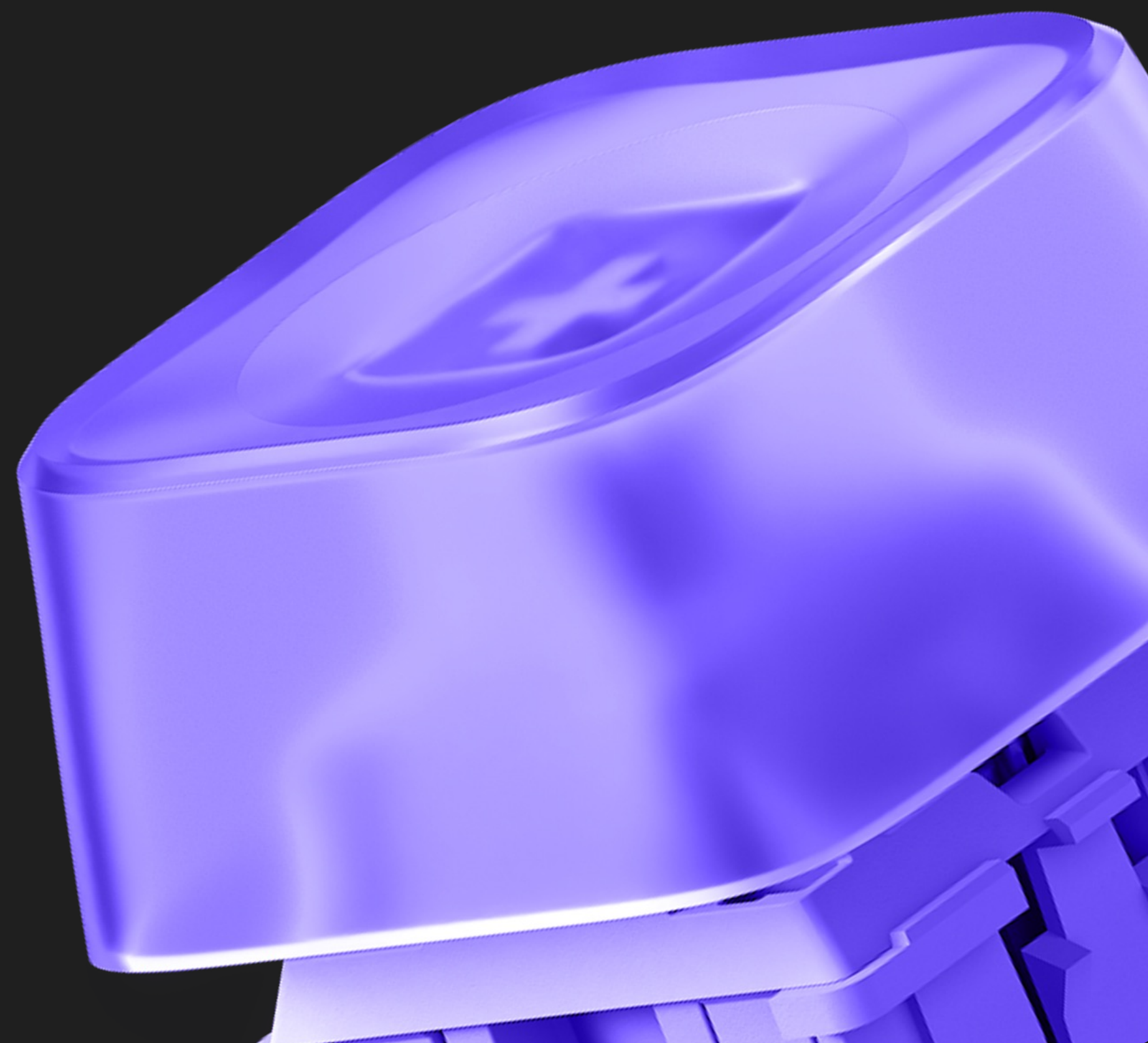
- Используйте единообразные имена директорий и файлов.
- Разделяйте исходный код, тесты, документацию и вспомогательные файлы.
- Добавляйте README с описанием цели проекта, зависимостей и инструкций по запуску.
- Храните конфигурацию окружения в проекте (*requirements.txt*, *pyproject.toml*, *Dockerfile* и т. д.).

# Структура проекта



- Используйте **.gitignore** для исключения временных файлов, кешей и локальных настроек.
- Организуйте тесты так, чтобы их можно было запускать одной командой.
- Документируйте структуру и принципы работы проекта в docs/ или wiki.
- Следите за чистотой корневой директории — минимум файлов вне каталогов.

# Работа в VS Code





# VS Code

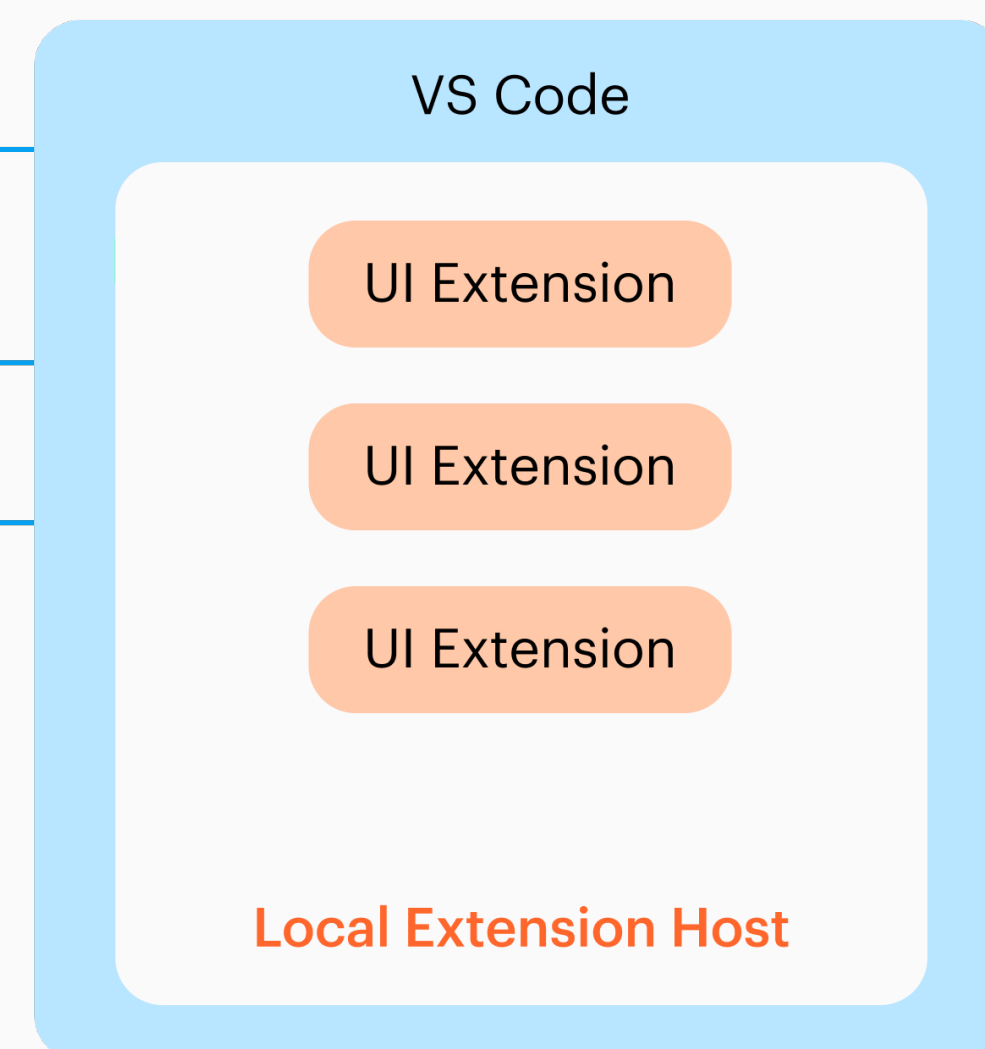
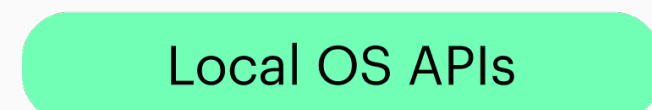
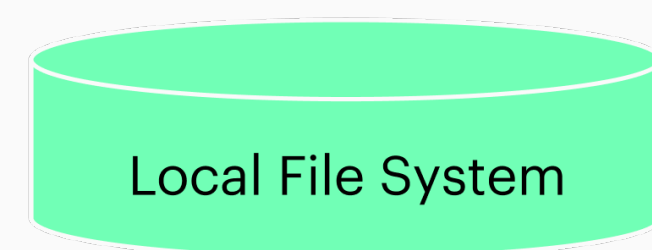
- Лёгкая и быстрая IDE с широкими возможностями настройки
- Поддержка множества языков и расширений (Python, Docker, и др.)
- Встроенная интеграция с Git и системами контроля версий
- Удобная работа с удалёнными машинами через Remote SSH
- Поддержка отладчика, терминала и управления виртуальными окружениями в одном окне



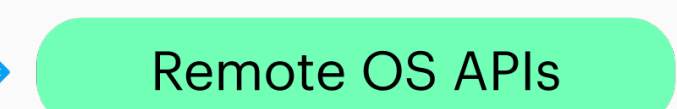
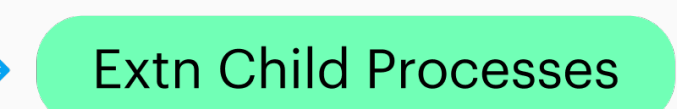
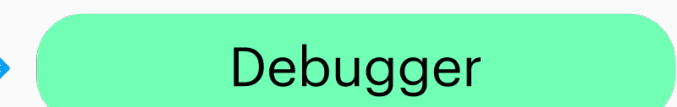
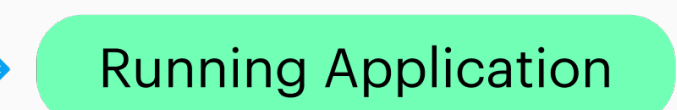
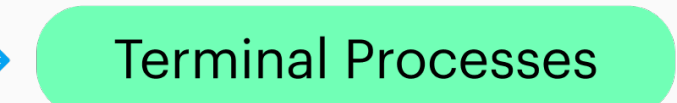
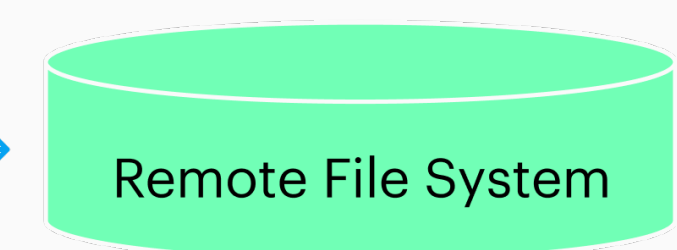
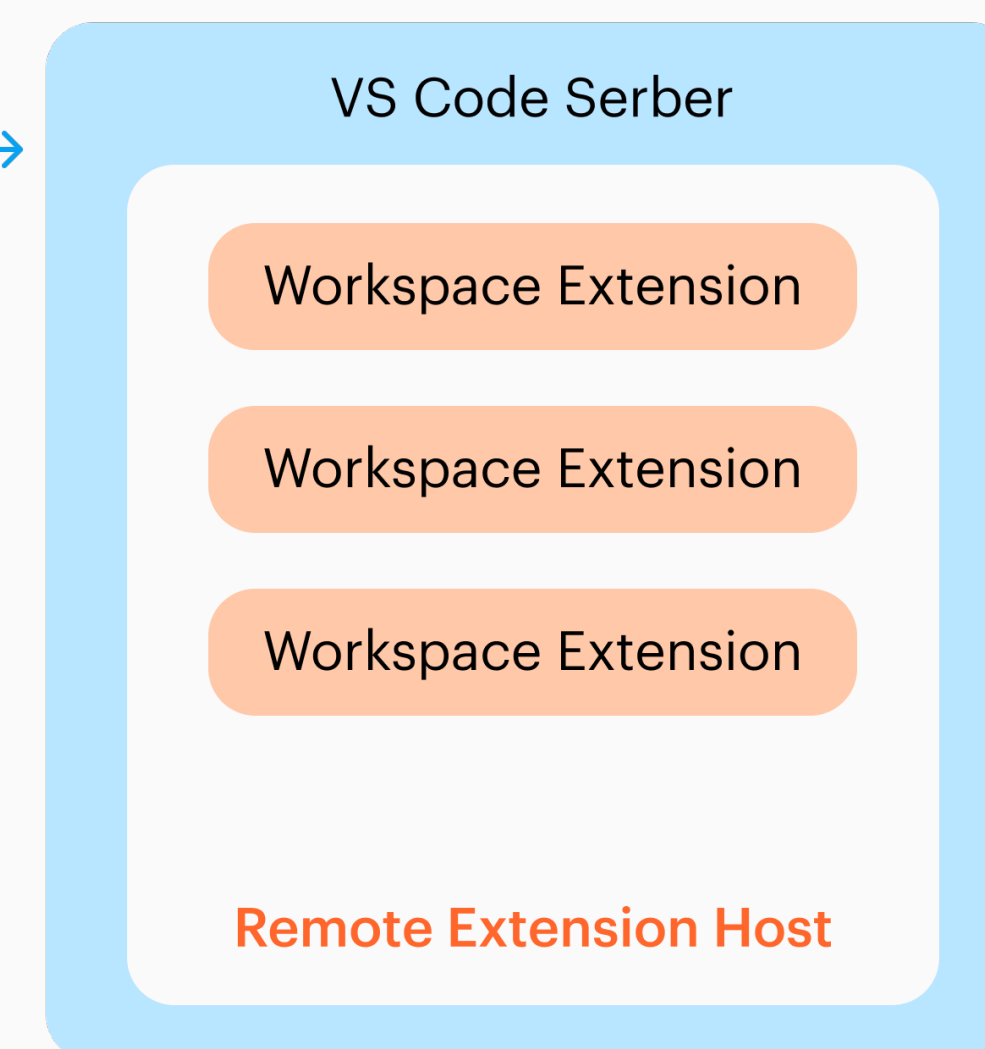
# Удалённая разработка в VS Code

**Remote-SSH** — это расширение для VS Code, которое позволяет подключаться к удалённому серверу через SSH и работать с файлами, как если бы они находились на локальной машине.

Local OS



Remote OS



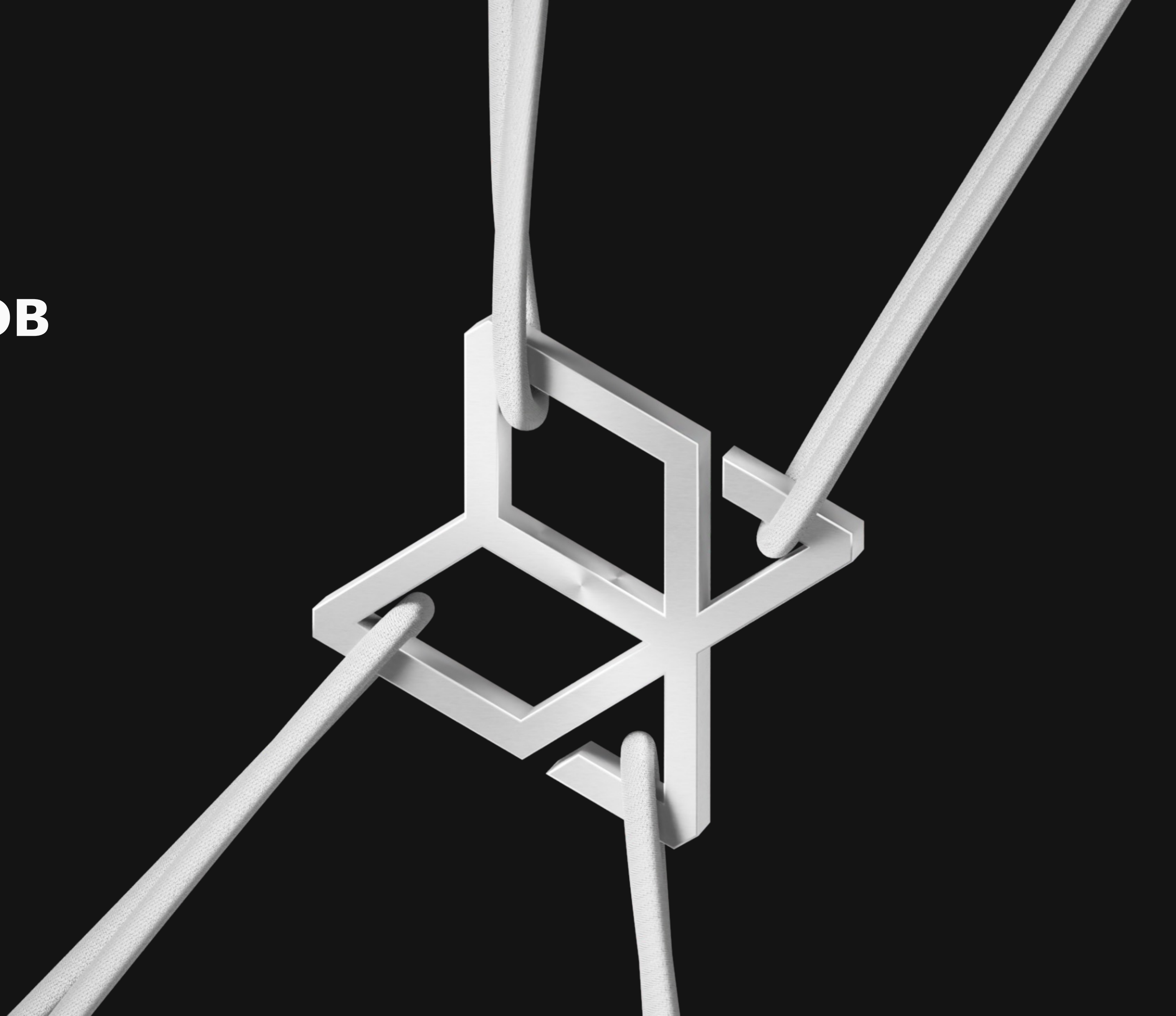


# Резюме



- Освоили основные команды Linux и работу в терминале.
- Научились подключаться к удалённым серверам через SSH.
- Разобрали основы Git и совместной разработки.
- Познакомились с принципами организации проекта.
- Рассмотрели VS Code как IDE для локальной и удалённой разработки.

# Организация Python-проектов





# Управление зависимостями: pip

- Стандартный менеджер пакетов для Python.
- Работает с PyPI (Python Package Index).
- Устанавливает, обновляет и удаляет пакеты.

**requirements.txt** — текстовый файл с зависимостями.

**requests==2.31.0**

**numpy>=1.25.0**

**jupyter**

```
# Установка пакета  
pip install requests
```

```
# Установка конкретной версии  
pip install requests==2.31.0
```

```
# Обновление пакета  
pip install --upgrade requests
```

```
# Установка из requirements.txt  
pip install -r requirements.txt
```

```
# Просмотр установленных пакетов  
pip list
```

```
# Сохранение зависимостей  
pip freeze > requirements.txt
```

```
# Удаление пакета  
pip uninstall requests
```

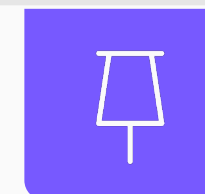
# Управление зависимостями: **pip**

## Недостатки **pip** как менеджера пакетов

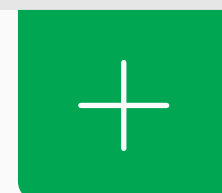
- Нет автоматического разрешения конфликтов зависимостей.
- Нет поддержки **lock-файлов** → сложно повторить окружение.
- Нет декларативного управления проектом (нет единого манифеста).

# Виртуальные окружения

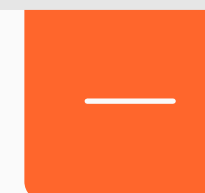
## Изолированная среда для Python-проекта:



- своя версия интерпретатора,
- свой набор библиотек.



- **Изоляция зависимостей** — у каждого проекта свой набор библиотек.
- **Предотвращение конфликтов версий** — разные версии пакетов могут сосуществовать.
- **Воспроизводимость** — легко запустить проект на другой машине.
- **Стабильность локальной среды** — защищаем системный Python от неожиданных изменений.



- **Не гарантируют production-среду** — окружение  $\neq$  контейнер (Docker решает).
- **Не изолируют системные библиотеки** — могут быть проблемы при нативных зависимостях.



# venv

## Базовый инструмент

Встроен в Python, используется как основа у более продвинутых менеджеров пакетов.

## Как устроено внутри?

- Копия Python-интерпретатора
- Каталоги `include/lib` с пакетами (`site-packages`)
- Скрипты для активации

```
# Создать окружение
python3 -m venv .venv

# Активировать окружение
source .venv/bin/activate

# Установить зависимости
pip install -r requirements.txt

# Сохранение зависимостей
pip freeze > requirements.txt

# Деактивировать окружение
deactivate
```

# Сборка и публикация пакетов

## Зачем нужна сборка?

- Упаковать проект в переносимый формат (wheel/sdist).
- Сделать пакет доступным для установки через pip/PyPI.

- **Ручная настройка** — метаданные и зависимости прописываются вручную.
- **Разделённые шаги** — сборка и публикация выполняются отдельно.
- **Нет стандарта** — разные проекты имеют разную структуру setup.py.

## Базовые инструменты

- **setup.py** + setuptools / wheel
- **twine** для публикации на PyPI

```
# Создать wheel
python setup.py sdist bdist_wheel

# Загрузить на PyPI
twine upload dist/*
```

# pyproject.toml

## Современный стандарт Python-проектов

### Зачем нужен?

- Единый файл конфигурации для проекта (PEP 518 / PEP 621).
- Содержит метаданные проекта, зависимости и настройки сборки (и не только).

### Проблемы, которые решает

- ✗ Разделение `setup.py` / `requirements.txt` / `MANIFEST.in`.
- ✗ Сложная сборка и публикация.
- ✗ Отсутствие единообразного формата для инструментов.
- ✓ Всё в одном файле, стандартизировано и удобно.

```
[project]
name = "my-service"
version = "0.1.0"
description = "Demo Python service"
authors = [
    { name="Ivan Ivanov", email="ivan@example.com"
    }
]
license = { text = "MIT" }
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.110.0,<0.111.0",
    "uvicorn[standard]>=0.22.0,<0.23.0",
    "requests>=2.31.0"
]

[tool.ruff]
line-length = 88
select = ["E", "F", "W", "C90"]
exclude = ["tests/fixtures", "build"]

[build-system]
requires = ["setuptools>=42", "wheel"]
build-backend = "setuptools.build_meta"
```



# Современные менеджеры пакетов: Poetry/uv

→ Инструменты «всё в одном» для Python-проектов.

→ Используют `pyproject.toml` как единый источник конфигурации.

- Управление виртуальными окружениями
- Установка и обновление зависимостей с разрешением конфликтов
- Lock-файлы для воспроизводимости окружений
- Сборка wheel/sdist и публикация на PyPI



# uv

- **Всё в одном** — окружение, зависимости, сборка, публикация, запуск сервисов и задач.
- **Использует `pyproject.toml`** — единый источник конфигурации проекта.
- **Lock-файл** — гарантирует воспроизводимость окружения.
- **Управление dev-зависимостями** — легко добавлять и обновлять dev-пакеты.
- **Высокая производительность** — написан на Rust и работает очень быстро.
- **Активное развитие и поддержка сообщества** — современный инструмент с растущим сообществом пользователей и разработчиков.

```
# Создать новый проект
uv init

# Установить новый пакет
uv add requests

# Удалить пакет
uv remove requests

# Обновить окружение проекта
uv sync

# Запустить скрипт/сервис
uv run my_service.main:app

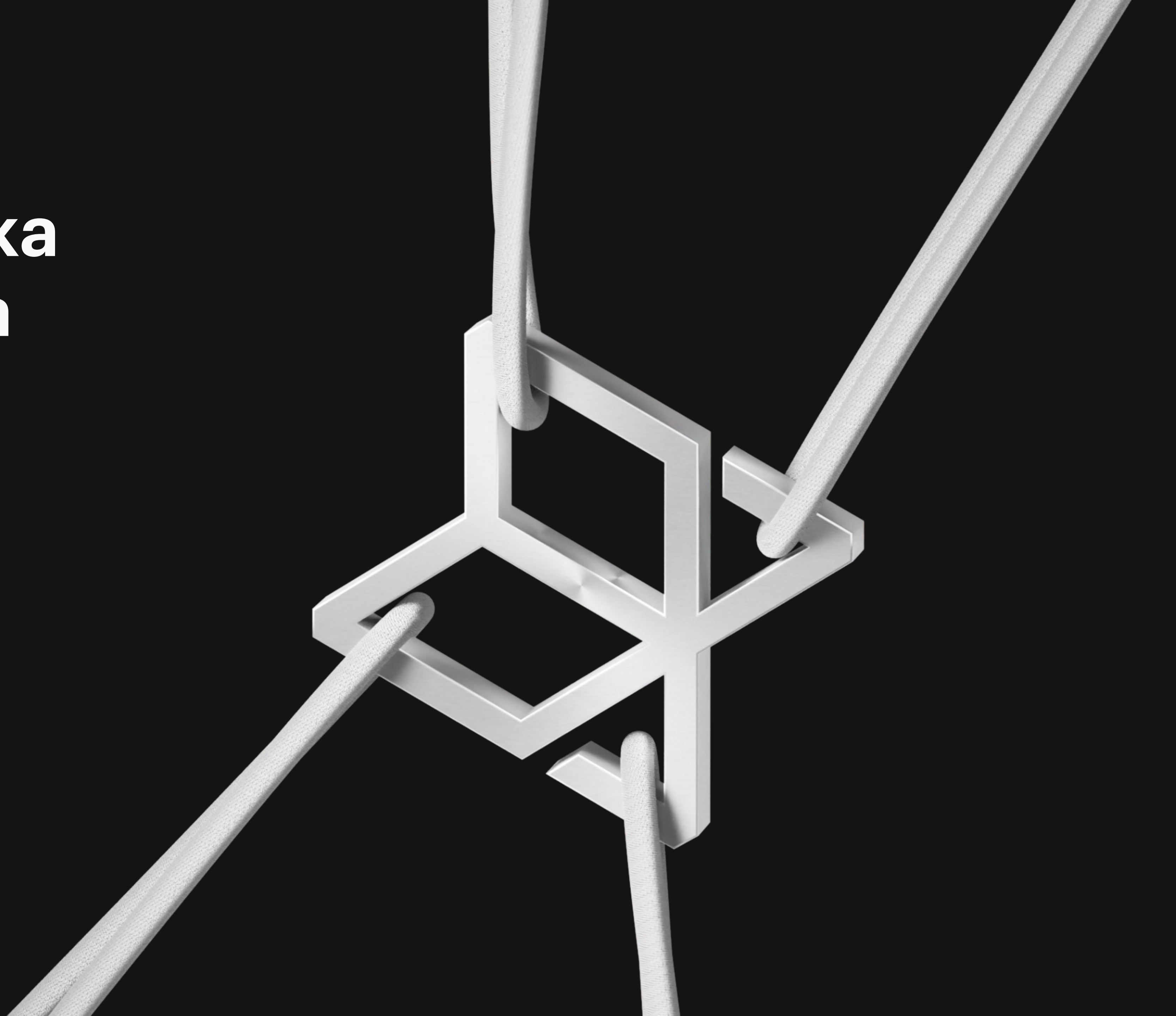
# Вывести дерево зависимостей
uv tree

# Собрать пакет
uv build

# Опубликовать на PyPI
uv publish

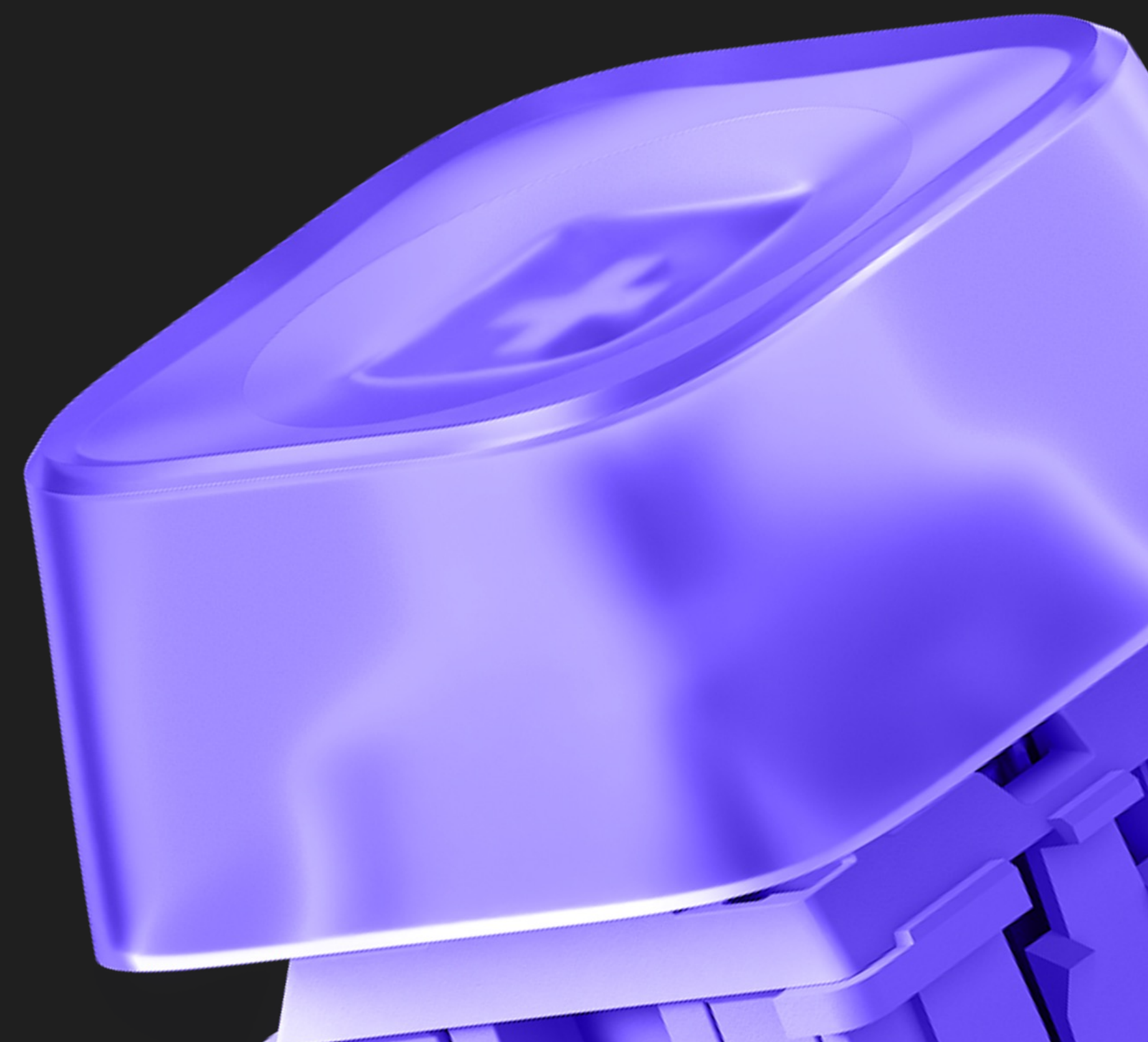
# Обновить lock-файл
uv lock
```

# Запуск и отладка Python-сервиса





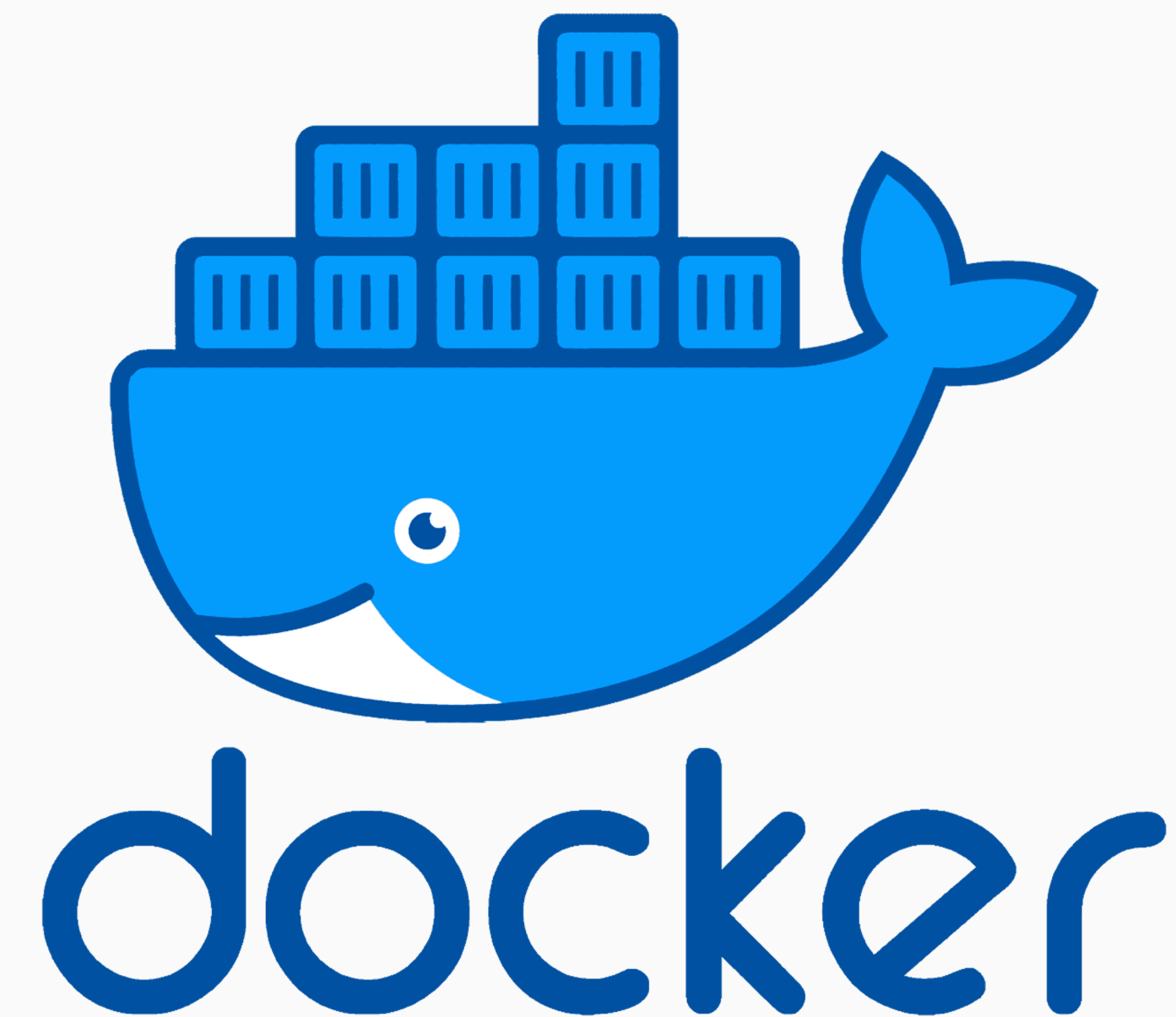
# Контейнеризация с Docker



# Docker

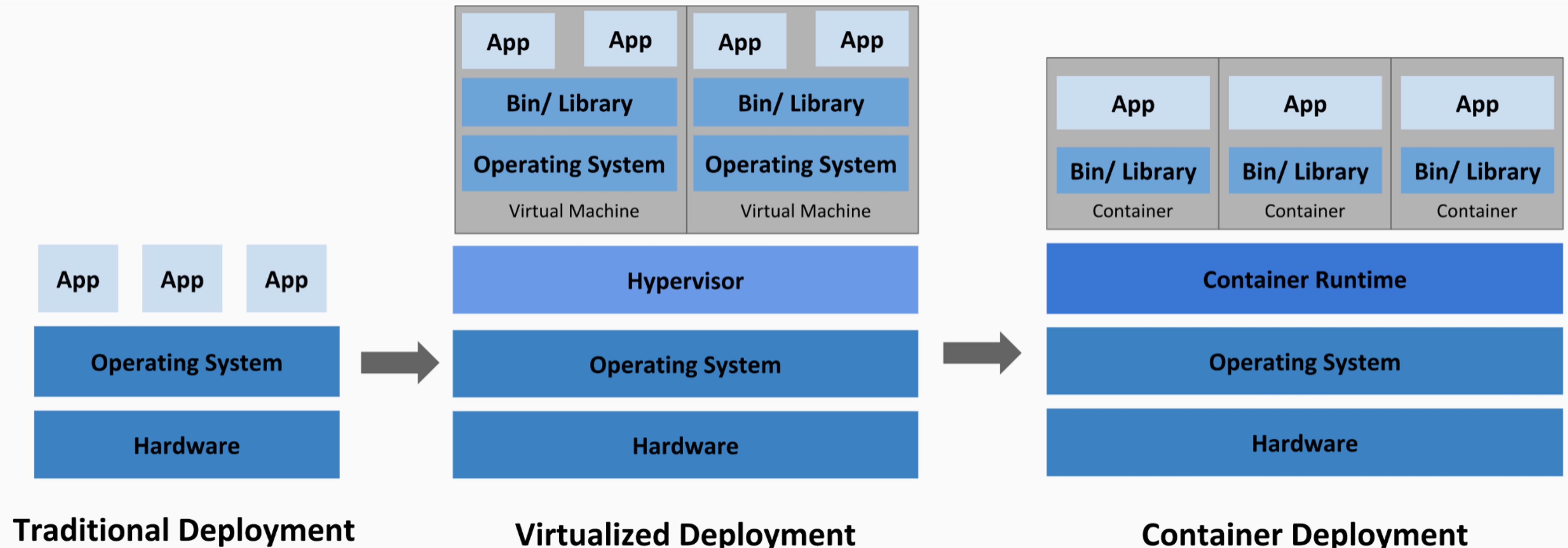
В современном мире разработки программного обеспечения одной из ключевых задач является обеспечение портативности, масштабируемости и изоляции приложений.

[Docker](#) — это платформа для контейнеризации, которая позволяет упаковывать, распространять и запускать приложения в стандартизированных средах, называемых контейнерами.





# Виртуализация и контейнеризация



# Виртуализация и контейнеризация

## ВИРТУАЛИЗАЦИЯ

### Плюсы

- Лучшая изоляция между приложениями.
- Можно запускать разные ОС на одном сервере.

### Минус

- Большие накладные расходы на ресурсы (каждая VM требует отдельную ОС).
- Более долгий запуск по сравнению с контейнерами.

## КОНТЕЙНЕРИЗАЦИЯ

### Плюсы

- Легковесность (нет необходимости дублировать ОС).
- Быстрое развёртывание и масштабирование.
- Меньшее потребление ресурсов.

### Минус

- Изоляция слабее, чем у виртуальных машин (так как используется общее ядро).
- Требуется совместимость с ОС хоста (нельзя запустить Windows-контейнеры на Linux без дополнительных решений).



# Причины популярности Docker

**Стандартизированное API** — единый интерфейс для работы с контейнерами, что упрощает автоматизацию и интеграцию в CI/CD-пайплайны.

**Полноценная экосистема** — Docker Hub и Private Registry для хранения и распространения образов.

**Упрощённая разработка и развёртывание** — одинаковые среды на любой ОС. Эффективная «слоистая» файловая система (layered filesystem).

**Масштабируемость** — интеграция с Kubernetes и другими оркестраторами.

**Мультиплатформенность** — работает на Windows, macOS и Linux.

# Основные сущности Docker

**Dockerfile** — это рецепт (инструкция) для создания образа.

- Описывает, какие зависимости и настройки нужны для контейнера.

## ПРИМЕР

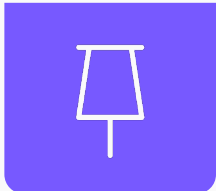
```
FROM python:3.9  
  
COPY app.py /app/app.py  
CMD ["python", "/app/app.py"]
```

- Используется для создания образа с помощью **docker build**.

**Образ (Image)** — это шаблон (слепок файловой системы), созданный из **Dockerfile**.

- Содержит все зависимости (библиотеки, исполняемые файлы, конфигурации).
- Образы хранятся в **Docker Registry** (например, **Docker Hub**).
- Запускается как контейнер с помощью **docker run**.

# Основные сущности Docker

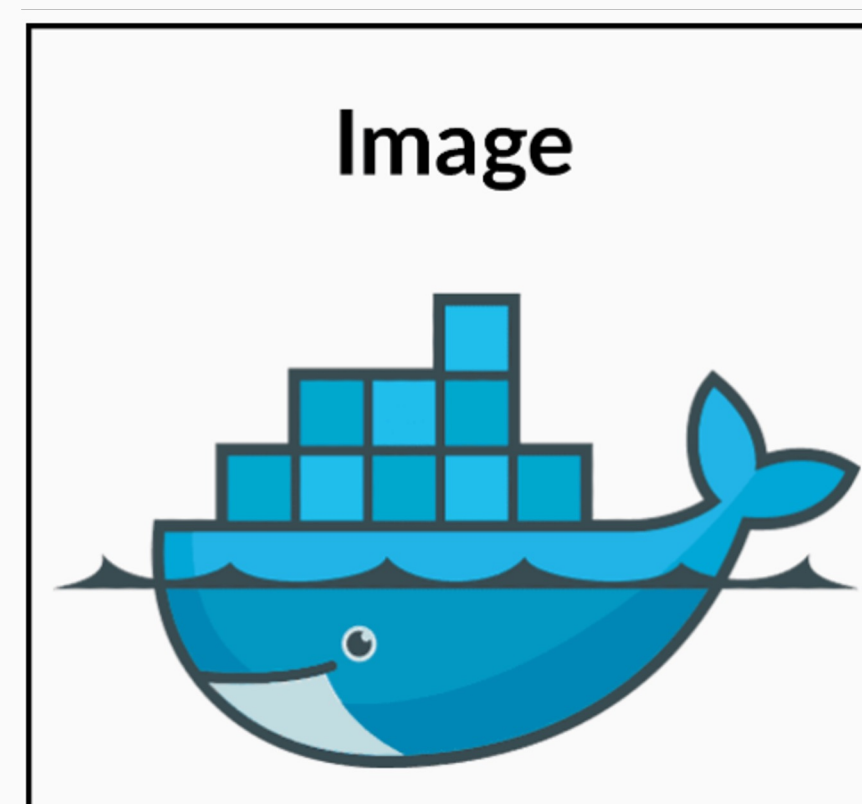


**Контейнер (Container)** — это запущенный экземпляр образа.

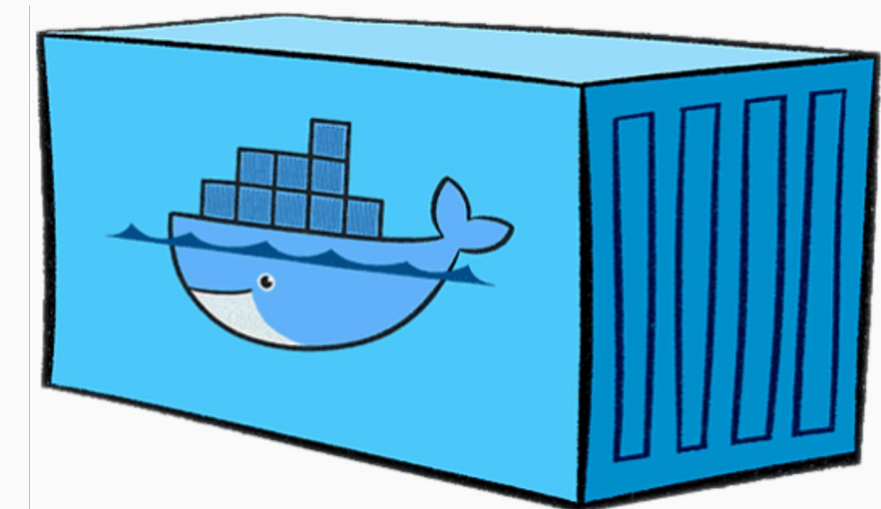
- Работает изолированно, но использует ресурсы хоста.
- Можно запускать, останавливать, удалять (**docker start**, **docker stop**, **docker rm**).

```
FROM example/rails
MAINTAINER example@example@gmail.com
# Add here your preinstall libs (e.g. imagemagick, mysql lib, ssh config)
# Install imagemagick
RUN apt-get update
RUN apt-get -qq -y install libmagickwand-dev imagemagick
# Install for mysql gem
RUN apt-get install -qq -y mysql-server mysql-client libmysqlclient-dev
# Install for Webahote
RUN apt-get install libssl0.9.8 -y
RUN apt-get install ttf-unfonts-core -y
# (required) Install Rails App
ADD Gemfile /app/Gemfile
ADD Gemfile.lock /app/Gemfile.lock
RUN bundle install --without development test
ADD . /app
# Overwrite unicorn
ADD config/unicorn.rb /app/config/unicorn.rb
FROM example/rails
MAINTAINER example@example@gmail.com
# Add here your preinstall libs (e.g. imagemagick, mysql lib, ssh config)
# Install imagemagick
RUN apt-get update
RUN apt-get -qq -y install libmagickwand-dev imagemagick
# Install for mysql gem
RUN apt-get install -qq -y mysql-server mysql-client libmysqlclient-dev
# Install for Webahote
RUN apt-get install libssl0.9.8 -y
RUN apt-get install ttf-unfonts-core -y
# (required) Install Rails App
ADD Gemfile /app/Gemfile
ADD Gemfile.lock /app/Gemfile.lock
RUN bundle install --without development test
ADD . /app
# Overwrite unicorn
ADD config/unicorn.rb /app/config/unicorn.rb
```

Dockerfile



Docker image



Docker Container



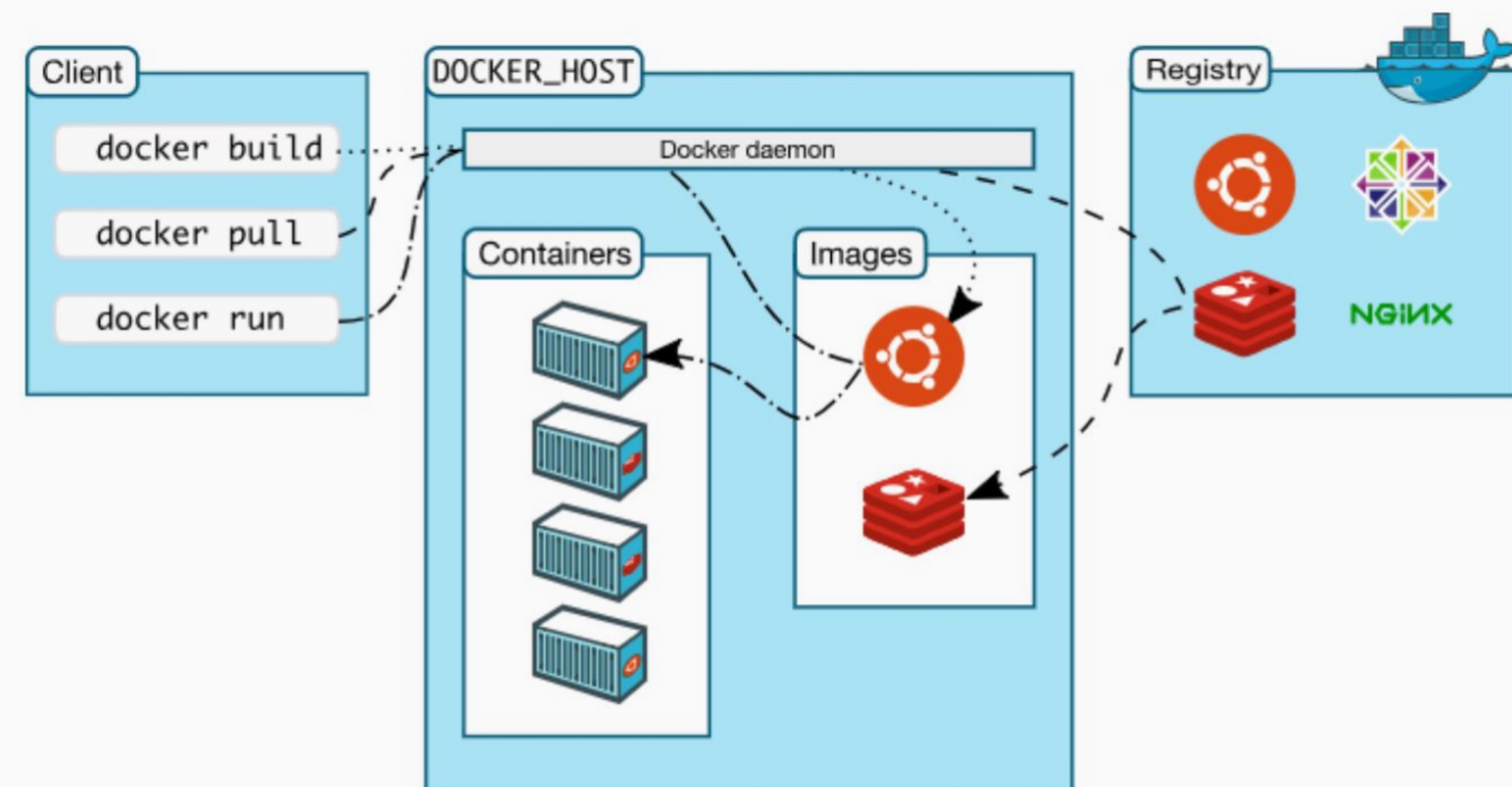
# Основные сущности Docker

**Docker Daemon** — фоновый процесс, управляющий контейнерами и образами.

→ Обработывает команды `docker build`, `docker run`, `docker pull`.

**Docker Registry** — репозиторий для хранения образов.

→ Позволяет загружать (`docker pull`) и отправлять (`docker push`) образы.



# Жизненный цикл контейнера

1. Написание: **Dockerfile** →
2. Сборка образа: **docker build** →
3. Загрузка образа в Registry: **docker push** (optional) →
4. Запуск контейнера: **docker run** →
5. Управление контейнером: **start/stop/restart/rm**

```
FROM example/rails

MAINTAINER example@example@gmail.com

# Add here your preinstall libs eg. imagemagick, mysql lib, ssh config
## Install imagemagick
RUN apt-get update
RUN apt-get -qq -y install libmagickwand-dev imagemagick

## Install for mysql gem
RUN apt-get install -qq -y mysql-server mysql-client libmysqlclient-dev

## Install for Webshots
RUN apt-get install libssl0.9.8 -y
RUN apt-get install ttf-unfonts-core -y

#(required) Install Rails App
ADD Gemfile /app/Gemfile
ADD Gemfile.lock /app/Gemfile.lock
RUN bundle install --without development test
ADD . /app

# Overwrite unicorn
ADD config/unicorn.rb /app/config/unicorn.rb

FROM example/rails

MAINTAINER example@example@gmail.com

# Add here your preinstall libs eg. imagemagick, mysql lib, ssh config
## Install imagemagick
RUN apt-get update
RUN apt-get -qq -y install libmagickwand-dev imagemagick

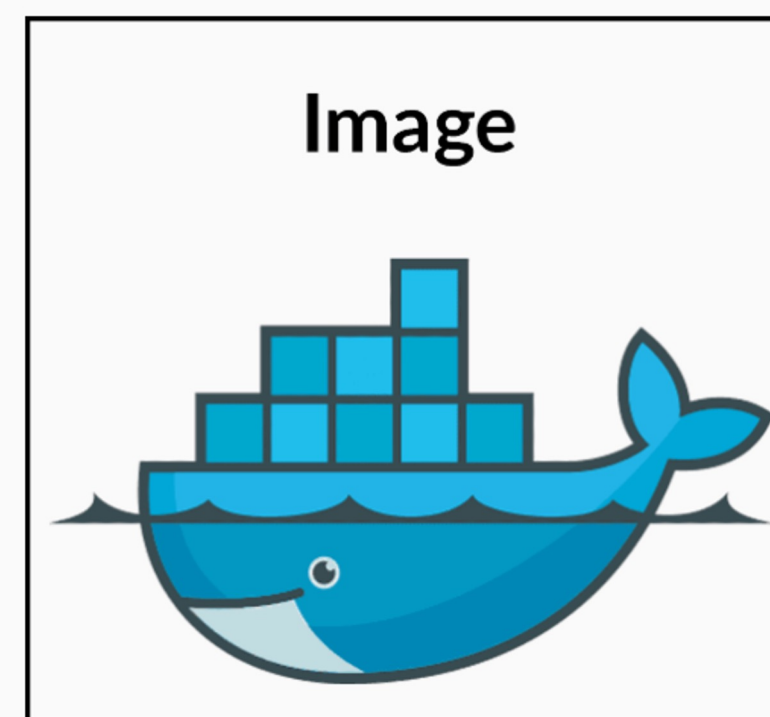
## Install for mysql gem
RUN apt-get install -qq -y mysql-server mysql-client libmysqlclient-dev

## Install for Webshots
RUN apt-get install libssl0.9.8 -y
RUN apt-get install ttf-unfonts-core -y

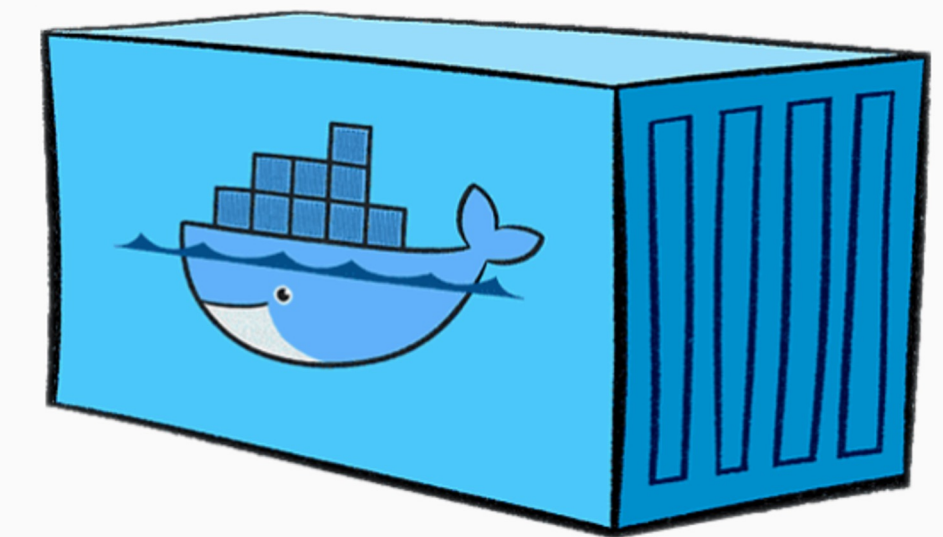
#(required) Install Rails App
ADD Gemfile /app/Gemfile
ADD Gemfile.lock /app/Gemfile.lock
RUN bundle install --without development test
ADD . /app

# Overwrite unicorn
ADD config/unicorn.rb /app/config/unicorn.rb
```

**Dockerfile**



**Docker image**



**Docker Container**



# Docker в разработке

01

## Среда разработки

- Один раз описали окружение (в Dockerfile/Compose) — одинаково работает у всех.
- Нет «У меня не запускается» из-за разных Python/Java/библиотек.

02

## Тестирование и CI/CD

- Легко поднимать сервисы в GitHub Actions, GitLab CI.
- Тесты проходят в том же окружении, что и продакшен.

03

## Продакшен-деплой

- Тот же образ запускается и локально, и на сервере, и в Kubernetes.
- Упрощает выкатывание новых версий: заменили контейнер, и готово.

04

## Микросервисы и изоляция

- Каждый сервис работает независимо, в отдельном контейнере.
- Легко контролировать ресурсы (CPU, RAM).
- Нет конфликтов зависимостей между сервисами.



# Взаимодействие контейнеров в Docker

## → Хост ↔ Контейнер

- Проброс портов (-p 8080:80) для доступа к сервисам контейнера с хоста.
- Монтирование директорий (-v ./data:/app/data) для обмена файлами между хостом и контейнером.

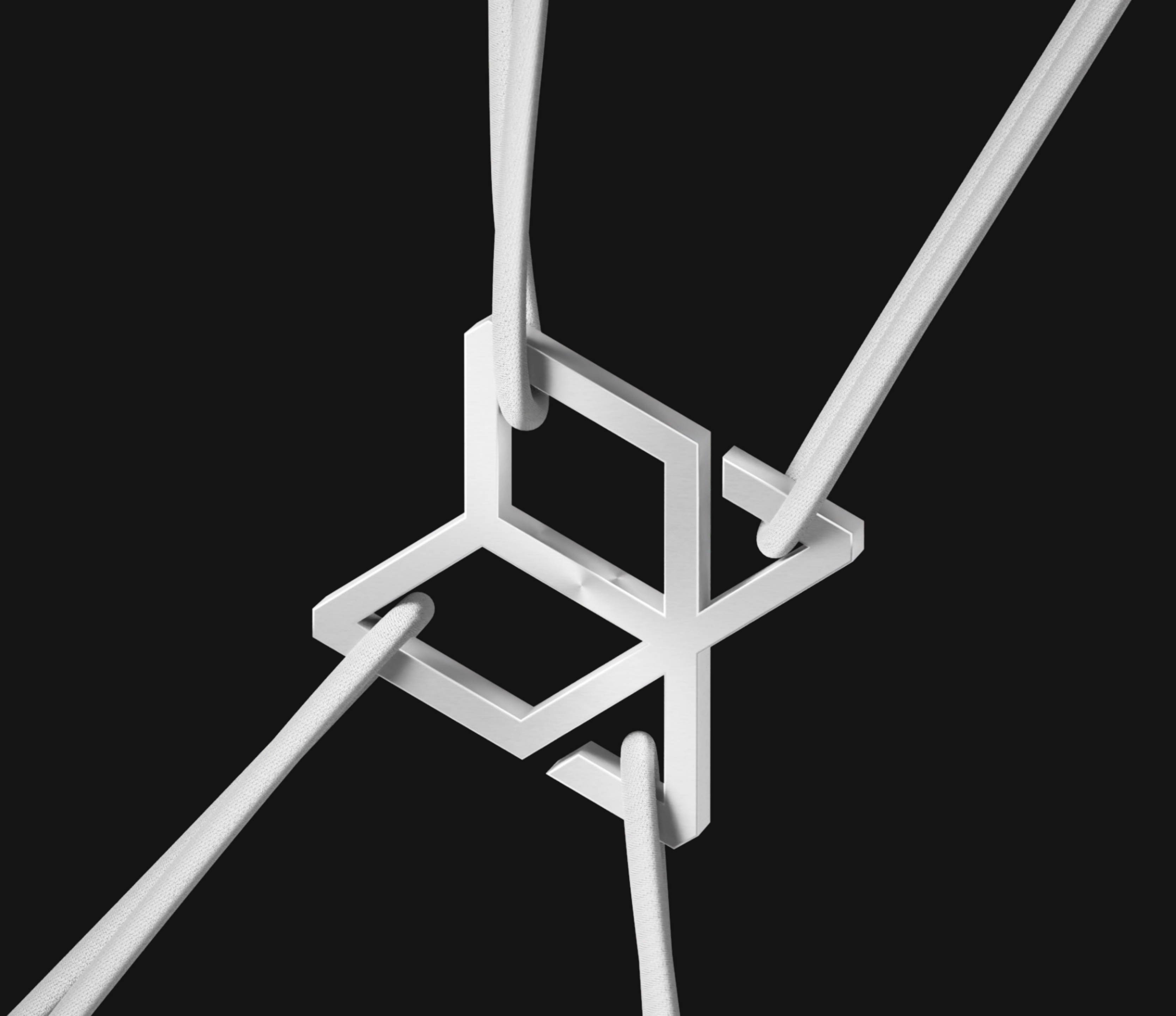
## → Контейнер ↔ Контейнер

- По умолчанию все контейнеры в дефолтной bridge-сети видят друг друга по IP.
- Для общения по именам сервисов создаются пользовательские сети (docker network create).

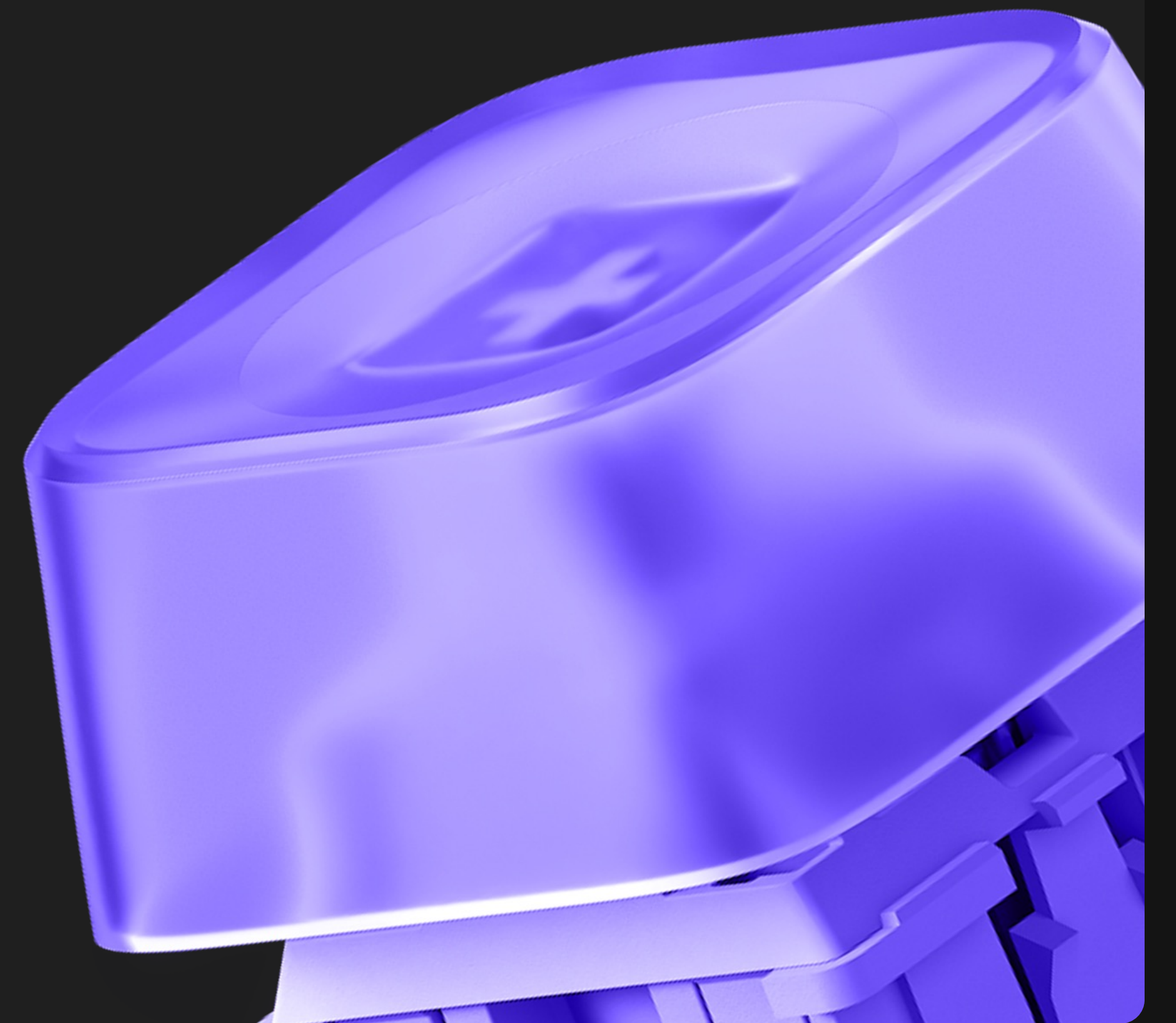
## → Контейнер ↔ Хост

- Из контейнера к хосту можно обратиться:
- через gateway дефолтной сети (172.17.0.1);
- host.docker.internal (Docker Desktop, новые версии Linux).

Практика



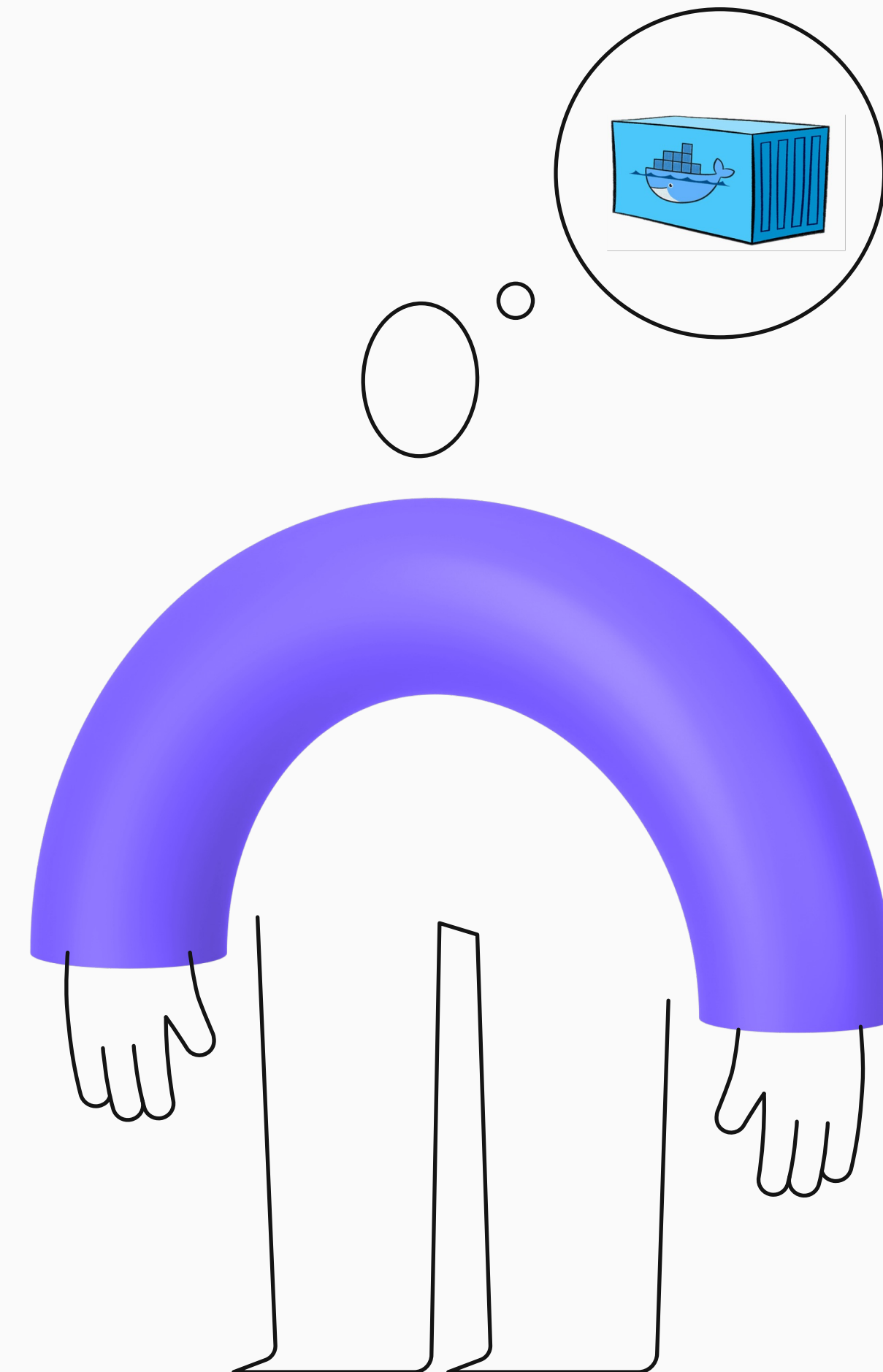
# Docker Compose





# Что такое Docker Compose

- Инструмент для описания, запуска контейнерных приложений.
- **Файл конфигурации: `compose.yaml`.**
  - Определяет **контейнеры (сервисы)**, которые будут запущены.
  - Задаёт их **настройки и взаимодействие** между собой.
- **Основная идея:** собрать весь стек (приложение + базу + кеш и др.) в одном файле, чтобы управлять им как единым целым.



# compose.yaml

- **services** — контейнеры приложения.
- **volumes** — постоянное (персистентное) хранение данных.
- **networks** — изоляция и взаимодействие контейнеров.

```
version: '3.9'

services:
  app:
    image: myapp:latest
    build: ./app
    ports:
      - "8080:80"
    networks:
      - backend
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - backend

volumes:
  db_data:

networks:
  backend:
```

# Основные команды Docker Compose

- `docker compose up` — поднять все сервисы (создать контейнеры и сети).
- `docker compose down` — остановить и удалить контейнеры, сети и тома.
- `docker compose build` — собрать образы сервисов.
- `docker compose logs` — просмотреть логи контейнеров.
- `docker compose exec <service> <command>` — выполнить команду внутри запущенного контейнера.

## Совет

- `up` с флагом `-d` запускает сервисы в фоновом режиме.
- `logs -f` позволяет «подписаться» на поток логов в реальном времени.



# Резюме



- Освоили работу с виртуальными окружениями Python и менеджерами пакетов.
- Разобрались с управлением зависимостями и конфигурацией Python-проекта.
- Познакомились с uv как инструментом для удобного управления проектом.
- Научились запускать и отлаживать Python-приложения, включая работу с VS Code.
- Рассмотрели основы Docker и контейнеризации приложений.
- На практике подняли мультиконтейнерное приложение с помощью Docker Compose.



Вопросы

